Chapter 16

Natural Language Processing in Finance

Every earlier chapter has worked with numbers that arrive already structured: prices, returns, ratios, transaction amounts. Yet much of the information that moves markets starts as an earnings-call transcript, a paragraph in a 10-K's risk factors, a news wire, or a social-media post. It must be converted into features before any of the tools of Chapters 6 through 15 apply. This chapter develops that conversion and the techniques built on it, collectively natural language processing (NLP).

Several earlier threads converge here. Chapter 1 noted that deep learning learns directly from unstructured text, and that large language models (LLMs) extend it to summarizing earnings calls and extracting information from filings. Chapter 3 observed that large asset managers use NLP to parse filings faster than human analysts. Chapter 6 introduced Naive Bayes as a generative classifier for word counts, of which LLMs are far more elaborate examples. Chapter 13 flagged that a sentiment score is only half of an event-driven strategy; executing the trade is the other half. This chapter makes those threads concrete.

Nineteen worked examples run through the chapter. The first build representations: bag-of-words and TF-IDF vectors for a few earnings-call sentences, and a sentiment score from a subset of the Loughran-McDonald dictionary, the lexicon used in finance research and practice. Those feed an event-driven trading signal, evaluated as Chapter 13 evaluates any strategy, and a Naive Bayes classifier that sorts 10-K risk-factor sentences into categories. The chapter then turns to modern architectures: word-embedding geometry, including a vector-arithmetic analogy, and a hand-computed scaled dot-product self-attention calculation, the core of the transformer, extended to show how a causal mask changes a token's representation across encoder-only, decoder-only, and encoder-decoder architectures. Then come a retrieval-augmented generation example grounding an answer in a retrieved passage, with precision@k and faithfulness metrics; a forward diffusion step for synthetic stress scenarios; a Gunning Fog readability comparison; a low-rank adaptation (LoRA) parameter count; a three-step LLM agent that computes and cross-checks a financial ratio; how per-step reliability compounds over a multi-step agent task; and an API-versus-self-hosting cost comparison showing when quantization's memory savings become deployment savings.

Because generative AI (LLMs, retrieval, prompting, and synthetic data from GANs and diffusion models) is developed here in more depth than any earlier topic, this chapter runs longer than the rest of the book.

16.1 Text as Data

Most financial information arrives as prose, and this part turns that prose into numbers a model can use.

16.1.1 Text as Data in Finance

Financial text differs from the numerical data used so far in three ways that matter for modeling. First, it is unstructured: a sentence has no predefined fields the way a balance sheet line does, so the first step is always to impose structure. Second, it is domain-specific: sentiment analysis trained on product reviews or social media misreads financial language. "Liability" is neutral accounting terminology, and "losses were smaller

Wulin.Suo@Queensu.ca

than expected" is good news phrased with two words, losses and smaller, that a generic model would likely score as negative. Third, financial text is produced under legal and strategic incentives: management chooses what to disclose, how to phrase it, and how much to hedge, so the text is a strategic object, not a neutral report.

Table 16.1 catalogs the main sources of financial text and where each is developed.

Source	Typical use	Developed in
Earnings call transcripts	Management tone and forward guidance, extracted as a sentiment score	Sections 16.1.3, 16.2.1
Regulatory filings (10-K, 10-Q)	Risk-factor classification, disclosure readability	Sections 16.2.2, 16.1.4
News wires and press releases	Event detection, sentiment scoring for event-driven strategies	Section 16.2.1
Social media and analyst reports	Aggregate sentiment signals, higher noise, higher volume	Section 16.3.4

Table 16.1: Sources of financial text and where they are used in this chapter

Figure 16.1 sketches the pipeline from raw text to a model-ready representation to an application. Every technique in this chapter fills in the middle step.

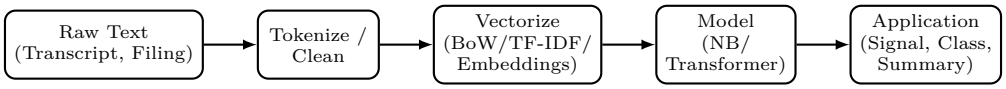


Figure 16.1: The text-to-application pipeline this chapter develops

The tokenize step decides what the units of text are. Classical methods split text into words at spaces and punctuation, so a word unseen in training is unknown. Transformer models instead use subword tokenization, most often byte-pair encoding (BPE) (Sennrich et al. 2016): start from single characters (or bytes) and repeatedly merge the most frequent adjacent pair into a new entry until the vocabulary reaches a target size, typically tens of thousands to a few hundred thousand entries. Common words become one token, while a rare one, a new ticker or a drug name, splits into familiar pieces, so no input is out of vocabulary. Numbers are where this matters in finance. A tokenizer chops a figure such as 1234567 into chunks set by merge frequencies, not place value, so the digit groups need not line up with thousands and millions. Singh and Strouse (2024) show the cost: merely forcing digits into groups of three from the right, as thousands separators do, raised GPT-4's accuracy on an addition task from 84.4% to 98.9%. The practical rule, applied in Section 16.5.3, is to treat a language model as a reader of numbers, not a calculator: let it extract figures, and hand the arithmetic to code.

16.1.2 From Text to Numbers: Bag-of-Words and TF-IDF

Suppose an analyst wants to feed thousands of earnings-call transcripts into one of Chapter 6's classifiers, each of which expects a fixed-length numeric vector, not a paragraph. The simplest conversion is the bag-of-words representation: fix a vocabulary, and represent each document by the count of each term it contains, discarding word order. This is drastic ("growth despite headwinds" and "headwinds despite growth" produce identical vectors), but it is a surprisingly effective baseline, and it is the "word counts" representation Chapter 6 anticipated for Naive Bayes.

A raw count, however, overweights common words that appear in almost every document and discriminate little. Term frequency-inverse document frequency (TF-IDF) corrects for this. For term t in document d, drawn from a collection of N documents,

$$\text{tf-idf}(t, d) = \text{tf}(t, d) \times \ln\left(\frac{N}{\text{df}(t)}\right), \tag{16.1}$$

Wulin.Suo@Queensu.ca

where $\text{tf}(t, d)$ is the raw count of t in d and $\text{df}(t)$ is the number of documents containing t at least once. A term that appears in every document has $\text{df}(t) = N$ and therefore an idf of $\ln(1) = 0$, its tf-idf weight vanishes regardless of how often it appears, while a term confined to a single document receives the largest possible idf weight.

Worked Example 16.1: Bag-of-Words and TF-IDF for Four Earnings-Call Sentences

Consider four short earnings-call sentences, treated as four documents ($N = 4$):

D_1: "revenue growth exceeded expectations this quarter"
D_2: "management is cautious about revenue growth given the headwinds"
D_3: "management remains confident despite the headwinds this quarter"
D_4: "expectations were not met and revenue declined"

Restricting attention to six vocabulary terms gives the term-frequency matrix in Table 16.2, alongside each term's document frequency, idf, and resulting tf-idf weight in D_1.

Term	D_1	D_2	D_3	D_4	df	idf $= \ln(4/\text{df})$	tf-idf in D_1
revenue	1	1	0	1	3	0.2877	0.2877
growth	1	1	0	0	2	0.6931	0.6931
headwinds	0	1	1	0	2	0.6931	0.0000
expectations	1	0	0	1	2	0.6931	0.6931
quarter	1	0	1	0	2	0.6931	0.6931
management	0	1	1	0	2	0.6931	0.0000

Table 16.2: Term frequency, document frequency, idf, and TF-IDF weight in D_1

The first four columns of Table 16.2 are the bag-of-words representation. Over the vocabulary (revenue, growth, headwinds, expectations, quarter, management) they give $D_1 = (1, 1, 0, 1, 1, 0)$ and $D_2 = (1, 1, 1, 0, 0, 1)$, the vectors a classifier would receive without weighting. They make the loss of word order concrete: any rearrangement of D_1's words produces the same vector, and D_1 and D_4 share two terms although one reports growth and the other a decline.

"Revenue" appears in three of the four documents, so it earns a modest idf of $\ln(4/3) \approx 0.2877$; every other term appears in two, giving a larger idf of $\ln(4/2) = \ln 2 \approx 0.6931$. "Headwinds" and "management" do not appear in D_1, so their tf-idf weight in D_1 is zero whatever their idf, while "growth," "expectations," and "quarter" each occur once in D_1 and contribute their full 0.6931. A downstream classifier, such as the Naive Bayes model of Section 16.2.2, would receive this in place of raw counts.

16.1.3 Financial Sentiment Analysis and the Loughran-McDonald Dictionary

Suppose a trader wants to know, within seconds of an earnings call ending, whether management sounded upbeat or defensive, without waiting for an analyst's summary. The most direct way to score sentiment is a lexicon-based approach: maintain a list of positive and negative financial words, count how many of each appear in a document, and combine the counts into a tone score,

$$\text{tone} = \frac{\#\text{positive words} - \#\text{negative words}}{\#\text{total words}}. \tag{16.2}$$

Generic lexicons, built from product reviews or general text, perform poorly in finance for the reason Section 16.1.1 raised: "tax," "cost," or "liability" read as negative in everyday English but are neutral in a filing. The Loughran-McDonald dictionary, built from 10-K filings, classifies words into finance-specific categories (positive, negative, uncertainty, litigious, and others) by how they function in financial disclosure.

Wulin.Suo@Queensu.ca

Worked Example 16.2: Scoring Two Earnings-Call Transcripts

Using a small subset of real Loughran-McDonald positive words ({strong, growth, exceeded, confident, improved}) and negative words ({cautious, headwinds, decline, declined, weak, concern, concerns}), consider two short transcripts:

> Transcript A: "revenue growth was strong this quarter and results exceeded expectations across every segment we remain confident heading into next year"
> Transcript B: "management is cautious about headwinds and results declined modestly we have some concerns about demand next quarter"

Transcript A contains 20 words, of which 4 (growth, strong, exceeded, confident) are positive and none are negative, giving a tone score of $(4-0)/20 = 0.20$. Transcript B contains 17 words, of which none are positive and 4 (cautious, headwinds, declined, concerns) are negative, giving a tone score of $(0-4)/17 \approx -0.2353$. Scores like these feed the event-driven trading signal developed next.

Worked Example 16.3: Scoring Central-Bank Communication

Company disclosures are not the only text markets parse by machine. The Federal Open Market Committee's post-meeting statement is arguably the most market-moving recurring document in finance, and it is scored with the machinery above, with one twist: the axis is not positive versus negative but *hawkish* versus *dovish*, language signaling tighter versus easier future monetary policy. The sign convention depends on the asset: a "strong" labor market is good news about the economy but hawkish news for bond prices. A general-purpose dictionary, and even Loughran-McDonald, therefore misreads central-bank language as generic lexicons misread 10-Ks, and practitioners maintain purpose-built hawkish and dovish word lists instead.

Using the hawkish list {strong, elevated, increases, tightening, restrictive} and the dovish list {moderated, eased, easing, patient, softening}, consider two consecutive stylized statements:

> January: "inflation remains elevated and the committee anticipates that ongoing increases in the target range will be appropriate the labor market remains strong"
> March: "inflation has moderated in recent months and the committee will be patient as it assesses incoming data the labor market has shown signs of softening"

The January statement contains 22 words, of which 3 (elevated, increases, strong) are hawkish and none are dovish, giving a net-hawkishness score of $(3-0)/22 \approx +0.1364$. The March statement contains 25 words, of which none are hawkish and 3 (moderated, patient, softening) are dovish, scoring $(0-3)/25 = -0.1200$. What a rates trader cares about, though, is neither level but the *change*: a drift of $-0.1200 - 0.1364 \approx -0.2564$, a decisive swing from hawkish to dovish. Every participant has already read the previous statement, so the information in a new one lies almost entirely in what changed. That is why coverage of an FOMC release is dominated by redline comparisons against the prior statement, and why the tone *difference* is the natural trading signal: the event-driven logic of the next section applied to the policy calendar. Research versions, scoring decades of central-bank communication and measuring the market reaction to its unanticipated component, are now a standard tool in empirical monetary economics (see Further Reading).

16.1.4 Disclosure Analysis: Readability and the Gunning Fog Index

Regulators require public disclosures to be understandable to ordinary investors (the U.S. Securities and Exchange Commission's "Plain English" rule is one such requirement), and readability metrics measure whether a passage meets that bar. The Gunning Fog index estimates the years of formal education a reader needs to understand a passage on a first reading,

$$\text{Fog} = 0.4 \left[\frac{\text{words}}{\text{sentences}} + 100 \times \frac{\text{complex words}}{\text{words}} \right], \tag{16.3}$$

where a complex word is (approximately) one with three or more syllables.

Wulin.Suo@Queensu.ca

Worked Example 16.4: Two Disclosure Paragraphs Compared

Consider two short passages. Paragraph A: "The firm's sales grew this year. Profits also rose. We expect this trend to grow next year." This has 17 words across 3 sentences and zero complex words (every word has one or two syllables), giving Fog $= 0.4\,[(17/3) + 100(0/17)] = 2.27$.

Paragraph B: "The Company's operations are subject to numerous regulatory, competitive, and macroeconomic uncertainties. These uncertainties could materially and adversely affect anticipated profitability and operational continuity." This has 24 words across 2 sentences, of which 14 are complex (numerous, regulatory, competitive, macroeconomic, uncertainties, materially, adversely, anticipated, profitability, operational, continuity, and others), giving Fog $= 0.4\,[(24/2) + 100(14/24)] = 28.13$.

A Fog index above roughly 17 corresponds to a post-graduate reading level; Paragraph B's 28.13 is far beyond that. The toy paragraphs are built to contrast sharply, but the direction matches a well-documented finding: 10-K risk-factor sections and other mandatory disclosures routinely score in this elevated range, dense, technical, and long-sentence.

Readability metrics give compliance teams an automatable way to flag disclosures that may miss the plain-English standard, and give researchers a candidate predictive signal (some published research finds that harder-to-read disclosures are associated with worse subsequent firm performance, consistent with strategic obfuscation).

The Fog index is also useful during drafting, not only after the fact. Rewriting Paragraph B in plainer language, "Our business faces many risks from rules, competition, and the economy. These risks could hurt our profits and how well we operate," gives 22 words across 2 sentences with only 3 complex words (competition, economy, operate), so

$$\text{Fog} = 0.4\,[(22/2) + 100(3/22)] \approx 9.85,$$

close to Paragraph A's score and nowhere near Paragraph B's original 28.13, while covering the same categories of risk and the same warning about profitability. This is the use a Plain English rule intends to encourage: the legal and business meaning did not change, only sentence structure and word choice, yet the Fog score fell by nearly two-thirds. A readability check built into the drafting workflow can therefore flag an overly complex passage before it is filed, not after a regulator or researcher measures it.

16.2 Classical Text Models

Before transformers, text was classified and traded on with methods that remain fast, auditable, and often sufficient.

16.2.1 Worked Example 16.5: An Event-Driven Sentiment Trading Signal

A sentiment score is only half of an event-driven strategy: the signal must still be evaluated like any trading strategy, and eventually converted into an executed position, which is Chapter 13's concern. This section stops at generating and evaluating the signal.

Suppose eight earnings-call events produce the tone scores and next-day stock returns in Table 16.3. A simple trading rule takes a long position when tone is positive and a short position when tone is negative, so the strategy's return on each event is sign(tone) $\times$ return.

The strategy is correct (its position's sign matches the return's) in 6 of 8 events, a 75% hit rate. Events 4 and 7 are misses: in event 4, mildly negative tone preceded a small gain, and in event 7, mildly positive tone a small loss. In both the tone score was close to zero and only weakly informative: a sentiment signal is noisy, not a deterministic predictor. Comparing the two strategies (Sharpe-like ratio: mean divided by standard deviation, with no annualization applied here),

$$\begin{aligned} \text{Signal-driven strategy:} &\quad \text{mean} = 0.775\%,\ \text{std} = 0.8481\%,\ \text{Sharpe-like} = 0.9138, \\ \text{Buy-and-hold:} &\quad \text{mean} = 0.075\%,\ \text{std} = 1.1829\%,\ \text{Sharpe-like} = 0.0634. \end{aligned}$$

The signal adds value on this small sample, with a higher and more consistent return, even though it is wrong a quarter of the time. Turning it into an executed position, at the right size and without excessive market impact, is the execution problem of Chapter 13.

Wulin.Suo@Queensu.ca

Event	Tone score	Next-day return (%)	Signal	Strategy return (%)	Correct?
1	+0.15	+1.2	Long	+1.2	Yes
2	−0.10	−0.8	Short	+0.8	Yes
3	+0.08	+0.3	Long	+0.3	Yes
4	−0.05	+0.4	Short	−0.4	No
5	+0.20	+2.1	Long	+2.1	Yes
6	−0.18	−1.5	Short	+1.5	Yes
7	+0.02	−0.2	Long	−0.2	No
8	−0.12	−0.9	Short	+0.9	Yes

Table 16.3: Eight earnings-call events: tone score, next-day return, signal, and strategy return

The tone score is a text-based sentiment measure, worth placing beside a price-based one: the CBOE Volatility Index (VIX) from Chapter 5, the market's "fear gauge," which aggregates option prices into a forward-looking measure of expected market-wide volatility without reading any text. The nickname needs care: VIX is mostly a forecast of how much the index will move, and only partly a reading of how investors feel, because an option's price compensates both for expected volatility and for the discomfort of bearing it. Only that second, risk-compensation component behaves like a sentiment score; the headline index blends the two. The measures are complementary. A tone score captures sentiment specific to one company's call, which VIX cannot see, while VIX moves on macroeconomic or geopolitical news that names no company and that a firm-specific model was never trained to detect. A more sophisticated strategy conditions on both, using VIX as a market-wide risk-regime signal, as Chapter 13 describes, beside the firm-specific tone score: a positive tone is a different proposition in a calm market than in one gripped by elevated uncertainty.

Worked Example 16.6: An Event Study: Abnormal Returns Around One Call

Table 16.3 judged the signal against raw next-day returns, which mix the stock's reaction to its own news with its ordinary co-movement with the market. On a day the market rises 1%, a stock with a beta above one rises too, whatever management said. The event study (Campbell et al. 1997) separates the two, and the evidence that text moves prices, including the research behind the Loughran-McDonald dictionary, is almost all stated in its terms. It has three steps. First, fit the market model

$$R_t = \alpha + \beta R_{m,t} + \varepsilon_t$$

over an estimation window that ends well before the event, so the news cannot contaminate the fit; here 120 trading days, days −130 to −11. Second, over a short event window, compute each day's abnormal return $AR_t = R_t - (\hat{\alpha} + \hat{\beta} R_{m,t})$, the part of the return the market does not explain, and add them into the cumulative abnormal return CAR. Third, test it: if the event carried no news, each AR_t is noise with the residual standard deviation $\hat{\sigma}$ of the fit, so over an L-day window

$$t = \frac{CAR}{\hat{\sigma}\sqrt{L}}, \tag{16.4}$$

which ignores the small additional error from estimating $\hat{\alpha}$ and $\hat{\beta}$ on 120 days.

Apply this to event 5 of Table 16.3, a tone score of +0.20 followed by a +2.1% return. The companion notebook simulates the estimation window from a market model with $\alpha = 0.02\%$, $\beta = 1.30$, and $\sigma = 1.40\%$ a day, so the fit can be checked against the truth: it returns $\hat{\alpha} = 0.0260\%$, $\hat{\beta} = 1.2949$, $\hat{\sigma} = 1.4215\%$, and $R^2 = 0.5022$. The call is held after the close on day 0, so the reaction falls on day +1, and the event window is days −1 to +1, with stock returns of −0.1%, 0.3%, and 2.1% and market returns of −0.2%, 0.4%, and 0.9% (expected returns computed from the unrounded estimates):

$$\begin{aligned}
\text{Day } -1: &\quad AR = -0.10 - (0.0260 + 1.2949 \times (-0.20)) = -0.10 - (-0.2330) = 0.1330\%,\\
\text{Day } 0: &\quad AR = 0.30 - (0.0260 + 1.2949 \times 0.40) = 0.30 - 0.5439 = -0.2439\%,\\
\text{Day } +1: &\quad AR = 2.10 - (0.0260 + 1.2949 \times 0.90) = 2.10 - 1.1914 = 0.9086\%,
\end{aligned}$$

Wulin.Suo@Queensu.ca

so $CAR(-1,+1) = 0.7977\%$, with standard error $1.4215\sqrt{3} = 2.4621\%$, $t = 0.324$, and a two-sided p-value of 0.7465.

Two lessons carry back to the trading signal. First, the market explains 56.7% of the 2.1% that Table 16.3 credited to tone: the stock's expected return on day +1, given the market's 0.9% rise and its beta of about 1.3, was already 1.19%, and the abnormal part is 0.91%. A signal scored on raw returns gets credit for every market rally that follows a positive call, so the hit rate and Sharpe-like ratio above should be recomputed on abnormal returns before anyone trades on them. Second, one event proves nothing. With a daily idiosyncratic volatility of 1.42%, even day +1's abnormal return alone has $t = 0.639$, and a three-day CAR of 0.80% is indistinguishable from noise. Averaging over N comparable events shrinks the standard error by $\sqrt{N}$: the same average CAR would give $t = 1.02$ over 10 events and $t = 2.05$ over 40. That is why published evidence on text and returns pools thousands of filings or calls, and why eight events cannot establish that a sentiment signal works.

16.2.2 Naive Bayes and Document Classification

Suppose a compliance team must sort thousands of 10-K risk-factor sentences into "Credit Risk" or "Market Risk," a task too large to do by hand each filing season and too nuanced for a keyword rule. Chapter 6 introduced Naive Bayes as a generative classifier that estimates

$$\Pr(y = k \mid x_1, \ldots, x_p) \propto \Pr(y = k) \prod_{j=1}^{p} \Pr(x_j \mid y = k) \tag{16.5}$$

and noted its suitability for text, where features are word counts and the conditional-independence assumption, while not literally true, distorts less than it might for correlated financial ratios. This section applies it using only the counts of six vocabulary words (default, borrower, counterparty, volatility, rate, currency).

Table 16.4 shows the six training sentences, three per class, written in the style of a 10-K risk-factors disclosure.

Sentence	Label
"A default by a major borrower or counterparty could trigger a chain of default across our loan portfolio."	Credit Risk
"Our borrower base includes several highly leveraged parties, and a default by any significant borrower could impair our credit portfolio."	Credit Risk
"We are exposed to counterparty risk if a significant counterparty were to default on its obligations under our agreements."	Credit Risk
"Significant volatility in interest rate markets, combined with broader market volatility, could adversely affect the value of our trading positions."	Market Risk
"Changes in the benchmark rate or the market rate environment, combined with volatility in trading conditions, could affect our net interest income."	Market Risk
"Fluctuations in the currency exchange market and adverse currency movements, together with changes in the base lending rate, could reduce our reported earnings."	Market Risk

Table 16.4: Training sentences for the 10-K risk-factor Naive Bayes classifier

Counting runs over the six vocabulary terms only, so the class total below is a vocabulary-token count, not a sentence length. Credit Risk contributes `default` four times, `borrower` three, and `counterparty` three, with the three market terms absent, giving $N_{\text{Credit}} = 10$; Market Risk contributes `rate` four times, `volatility` three, and `currency` twice, with the three credit terms absent, giving $N_{\text{Market}} = 9$. Chapter 6's Section 6.5.2 Laplace (add-one) smoothing correction then divides by $N_c + V$ with vocabulary size $V = 6$,

Wulin.Suo@Queensu.ca

that is by 16 for Credit Risk and 15 for Market Risk, giving the estimated word probabilities

$$\begin{aligned}
\Pr(\text{default} \mid \text{Credit}) &= 0.3125,\\
\Pr(\text{borrower} \mid \text{Credit}) = \Pr(\text{counterparty} \mid \text{Credit}) &= 0.25,\\
\Pr(\text{volatility} \mid \text{Credit}) = \Pr(\text{rate} \mid \text{Credit}) = \Pr(\text{currency} \mid \text{Credit}) &= 0.0625,\\
\Pr(\text{rate} \mid \text{Market}) &= 0.3333,\\
\Pr(\text{volatility} \mid \text{Market}) &= 0.2667,\\
\Pr(\text{currency} \mid \text{Market}) &= 0.20,\\
\Pr(\text{default} \mid \text{Market}) = \Pr(\text{borrower} \mid \text{Market}) = \Pr(\text{counterparty} \mid \text{Market}) &= 0.0667,
\end{aligned}$$

reflecting each class's training vocabulary.

Worked Example 16.7: Classifying an Ambiguous Sentence by Hand

Consider the held-out test sentence "A default by a key counterparty during a period of heightened market volatility could be exacerbated by continued volatility in broader financial markets," with word counts (default: 1, counterparty: 1, volatility: 2, all others: 0), an ambiguous case mentioning credit-risk terms and, twice, a market-risk term. With equal class priors $\Pr(\text{Credit}) = \Pr(\text{Market}) = 0.5$, the (unnormalized) class scores are

$$\begin{aligned}
\text{score}(\text{Credit}) &= 0.5 \times 0.3125^1 \times 0.25^1 \times 0.0625^2 = 0.00015259,\\
\text{score}(\text{Market}) &= 0.5 \times 0.0667^1 \times 0.0667^1 \times 0.2667^2 = 0.00015802.
\end{aligned}$$

Normalizing gives $\Pr(\text{Credit} \mid \text{doc}) \approx 0.4912$ and $\Pr(\text{Market} \mid \text{doc}) \approx 0.5088$: the model predicts Market Risk, narrowly and incorrectly, since the true label is Credit Risk. In such near-toss-ups the independence assumption matters most: the two "volatility" mentions tip a mixed-signal sentence to the wrong side by less than two percentage points of posterior probability.

Applying the same model to a test set of six sentences (three of each true class) produces the confusion matrix in Table 16.5.

	Predicted Credit	Predicted Market
Actual Credit	2 (TP)	1 (FN)
Actual Market	0 (FP)	3 (TN)

Table 16.5: Confusion matrix for the 10-K risk-factor Naive Bayes classifier (positive class: Credit Risk)

Precision is $2/2 = 1.00$, recall is $2/3 \approx 0.6667$, and $F_1 \approx 0.80$: every sentence labeled Credit Risk was Credit Risk, but the model missed one, the ambiguous sentence worked through above.

16.2.3 Word Embeddings: From Sparse Counts to Dense Vectors

Bag-of-words and TF-IDF vectors are sparse and high-dimensional: with a vocabulary of tens of thousands of words, most entries in any document's vector are zero, and nothing records that "stock" and "equity" are related while "stock" and "bond" are not: every word is a separate dimension. An embedding is a fixed-length, continuous-valued vector standing in for a discrete object, here a word, chosen so that geometric closeness reflects closeness in meaning. Word embedding methods (Word2Vec and GloVe are the classical examples) learn a dense, low-dimensional vector for each word from co-occurrence patterns in a large corpus, so that words used in similar contexts get similar vectors. Cosine similarity,

$$\cos(u, v) = \frac{u \cdot v}{\|u\| \, \|v\|} \tag{16.6}$$

is the standard way to measure how similar two embedding vectors are.

Wulin.Suo@Queensu.ca

Worked Example 16.8: Cosine Similarity of Toy Embeddings

To illustrate the geometry without training a model, assign six financial terms small hand-chosen two-dimensional vectors:

$$\begin{aligned} \text{stock} &= (0.90, 0.20), & \text{equity} &= (0.85, 0.25), \\ \text{bond} &= (0.10, 0.90), & \text{debt} &= (0.15, 0.85), \\ \text{bullish} &= (0.80, -0.30), & \text{bearish} &= (-0.75, -0.35). \end{aligned}$$

Computing cosine similarity between related pairs gives $\cos(\text{stock}, \text{equity}) \approx 0.9977$ and $\cos(\text{bond}, \text{debt}) \approx 0.9980$, both close to 1: the vectors point in nearly the same direction. Unrelated or opposed pairs differ: $\cos(\text{stock}, \text{bond}) \approx 0.3234$ is only weakly similar, and $\cos(\text{bullish}, \text{bearish}) \approx -0.7000$ is strongly negative, capturing near-opposites. In an embedding trained on financial text, the same structure, related concepts clustering and opposites pointing apart, emerges from co-occurrence alone, without the model being told what any word means.

A more striking property is that vector *arithmetic* can capture relationships between words, not just similarity, and this toy example was built so that the classic "analogy" relation holds exactly. Computing stock − equity + bond,

$$(0.90, 0.20) - (0.85, 0.25) + (0.10, 0.90) = (0.15, 0.85) = \text{debt},$$

reproduces the debt vector exactly, because the near-synonym pairs stock/equity and bond/debt were placed at the identical offset, $(0.05, -0.05)$. In a trained embedding space, the same arithmetic famously (if imperfectly) captures "king − man + woman ≈ queen." In a financial space, an analogous relation might hold between a country and its currency, letting an analyst query by relationship rather than single-word similarity. Real embeddings only approximate this, and the approximation is weaker for rare or abstract financial terms, but the toy example isolates the geometric idea before the noise of a high-dimensional trained model obscures it.

16.3 Transformers and Large Language Models

Attention changed what a language model can represent, and this part develops it from the mechanism up to retrieval and prompting.

16.3.1 Attention and the Transformer Architecture

Embeddings give each word a fixed vector regardless of context, but the same word can mean different things in different sentences ("the Fed will likely raise rates" versus "the flood raised water levels"). Transformer models address this with self-attention. Each token's embedding x is linearly projected, using three weight matrices W^Q, W^K, W^V learned in training and shared across every token, into a query vector $Q = xW^Q$, a key vector $K = xW^K$, and a value vector $V = xW^V$: the same embedding viewed through three learned lenses. Intuitively, a token's query asks "what am I looking for," its key advertises "what I offer to a token looking for this," and its value is the information it contributes once attention focuses on it: a search query, an index entry, and a payload.

A token's contextual representation is then a weighted combination of every token's value vector, with weights determined by how well each token's key matches the query,

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \qquad \text{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}, \tag{16.7}$$

where d_k is the key dimension, included to keep dot products from growing too large as d_k increases, and softmax rescales raw scores into positive weights summing to one, so the combination of value vectors is an interpretable average rather than an arbitrary weighted sum.

This mechanism, applied across many layers and attention "heads," is the core building block of BERT, GPT, and every other modern transformer language model. Figure 16.2 diagrams the three-projection, one-combination structure every attention layer repeats.

Wulin.Suo@Queensu.ca

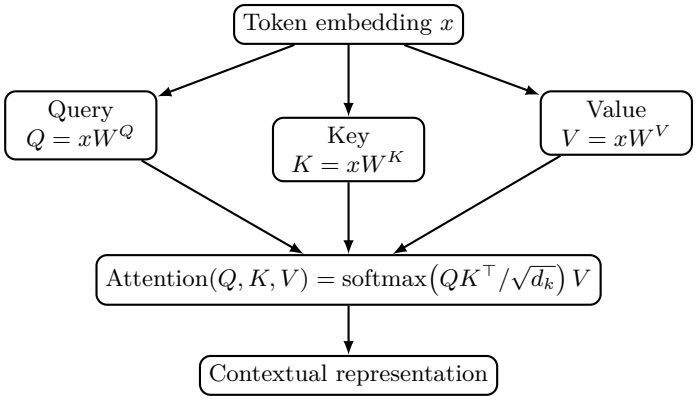


Figure 16.2: The self-attention computation as a diagram

Worked Example 16.9: Self-Attention for a Three-Token Sentence

Consider the toy sentence "stocks rallied today," with $d_k = 2$. Rather than training projection matrices W^Q, W^K, W^V, this example hand-chooses the resulting keys and values, standing in for what a trained model would produce: $K_{\text{stocks}} = (1, 0)$, $K_{\text{rallied}} = (0, 1)$, $K_{\text{today}} = (1, 1)$, and $V_{\text{stocks}} = (2, 0)$, $V_{\text{rallied}} = (0, 2)$, $V_{\text{today}} = (1, 1)$. "stocks" and "today" share the same first key component, so a query matching on that component attends to both equally, the effect this example illustrates.

The representation of "stocks" uses its query $Q_{\text{stocks}} = (1, 0)$. The raw scores $Q \cdot K_i/\sqrt{2}$ are 0.7071 against "stocks," 0 against "rallied," and 0.7071 against "today," since "rallied"s key is orthogonal to the query. Softmax turns these into attention weights of 0.4011, 0.1978, and 0.4011, which sum to 1, and the weighted sum of value vectors is

$$0.4011\,(2, 0) + 0.1978\,(0, 2) + 0.4011\,(1, 1) = (1.2033,\ 0.7967).$$

Figure 16.3 draws these weights, with edge thickness proportional to each token's contribution. "Today," a different word, contributes exactly as much to the new representation of "stocks" as "stocks" itself, because both keys align with the query, while "rallied" contributes far less, its value $(0, 2)$ pulled in only weakly.

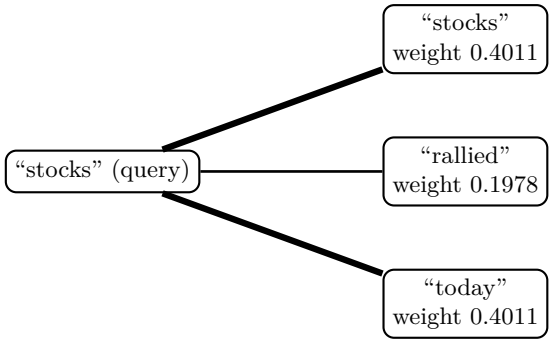


Figure 16.3: Self-attention weights for the query token "stocks" over the three-token sentence "stocks rallied today," with edge thickness proportional to attention weight

This mechanism lets a transformer represent "raise" differently next to "rates" than next to "water levels": a trained W^K would align "rates" and "raise" in a financial context and "water" and "raise" in an environmental one, as this example's keys were aligned by construction.

Multi-Head Attention and Positional Encoding

A single attention calculation forces every token to distribute its focus by one notion of relevance, but relevance is not one-dimensional: local grammatical structure (which word modifies which noun) and long-

Wulin.Suo@Queensu.ca

range dependency (which earlier passage a later reference points back to) are different relationships, and one attention pattern cannot represent both well. Two refinements give the mechanism used in practice. First, a transformer layer computes several attention calculations in parallel, called attention heads, each with its own learned Q, K, V projections, and concatenates their outputs. One head might attend mainly to nearby tokens (local syntax), another to the subject of a distant verb (long-range dependencies, such as linking a hedge described early in a paragraph to a number reported much later), and the model combines these perspectives. Second, the calculation above treats the input as an unordered set: nothing in $Q \cdot K$ depends on position, so "the bank raised rates" and "rates raised the bank" would receive identical weights given identical token embeddings. Positional encoding (PE) fixes this by adding a position-dependent vector to each embedding before attention, most commonly

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

and

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

for position pos and embedding dimension index i, so that word order survives into a mechanism that would otherwise discard it.

16.3.2 Encoder-Only, Decoder-Only, and Encoder-Decoder Architectures

Self-attention is the building block of every transformer, but different financial NLP tasks arrange it differently, and the three resulting architecture families are not interchangeable. An encoder-only model (BERT and its financial variant FinBERT are the standard examples) lets every token attend bidirectionally to every other token, the unmasked computation above, and is trained with a masked-language-modeling objective (predicting a hidden word from its context in both directions). This makes encoder-only models well suited to classification and embedding tasks, such as the sentiment scoring of Section 16.1.3 and the document classification of Section 16.2.2, where the full passage is available at once and the output is a label or vector, not new text. A decoder-only model (the GPT family and the LLMs of the rest of this chapter) restricts each token to attending to itself and earlier tokens, a causal mask, since it is trained to predict each next token from what precedes it, the one-token-at-a-time process of Section 16.3.4. An encoder-decoder model (T5-style architectures) uses both: a bidirectional encoder reads the input, and a causal decoder generates the output while cross-attending to the encoder's output, a natural fit for input-to-output tasks such as translating a filing or condensing an earnings call into a summary.

Worked Example 16.10: How a Causal Mask Changes a Token's Representation

Return to "stocks rallied today" and compute the representation of "rallied" with its query $Q_{\text{rallied}} = (0, 1)$, first with full bidirectional attention (as an encoder-only model would) and then with a causal mask hiding the later token "today" (as a decoder-only model would). Bidirectionally, the raw scores $Q_{\text{rallied}} \cdot K_i/\sqrt{2}$ are 0 against "stocks," 0.7071 against "rallied," and 0.7071 against "today," giving softmax weights of 0.1978, 0.4011, and 0.4011 and an output of

$$0.1978\,(2, 0) + 0.4011\,(0, 2) + 0.4011\,(1, 1) = (0.7967,\ 1.2033).$$

Under a causal mask, "today" is removed, leaving the scores against "stocks" (0) and "rallied" (0.7071); renormalizing these two via softmax gives weights of 0.3302 and 0.6698, and an output of

$$0.3302\,(2, 0) + 0.6698\,(0, 2) = (0.6605,\ 1.3395).$$

The same word in the same sentence gets two representations, $(0.7967, 1.2033)$ versus $(0.6605, 1.3395)$, purely from architecture: an encoder-only model's "rallied" may reflect that "today" follows it, while a decoder-only model's, used to predict what comes *after* "rallied," cannot peek at "today" without trivializing its training objective.

So swapping a decoder-only LLM into a task an encoder-only model was designed for, or vice versa, is not a matter of using "a big enough model": the two produce different token representations by construction, not by training data.

Wulin.Suo@Queensu.ca

16.3.3 How These Models Are Trained, and Why They Eventually Need Retraining

No one hand-writes the rule that tells a model which distant word a pronoun refers to, or which number a sentence's context makes plausible; the model must learn it, and the only training signal available at that scale is the text itself, with no human labels. None of the weight matrices in Figure 16.2, W^Q, W^K, W^V, or the many more in a transformer's feedforward and output layers, is hand-specified; each is learned by ordinary gradient descent (Chapter 6's Section 6.2.3), applied to a self-supervised objective computed from raw, unlabeled text at enormous scale.

An encoder-only model is pretrained by masking a random fraction of tokens and predicting each from its bidirectional context; a decoder-only model is pretrained on the single objective of predicting each next token from everything before it, the causal-masked computation just carried out by hand, repeated across a very large corpus. No human labels the output token by token; the text supplies the signal, which is why these models can be pretrained on far more data than any supervised model in this book.

Pretraining is not, however, a one-time event. Financial language drifts: new instruments, regulations, tickers, and company names appear, established terms acquire new meanings (a "pandemic bond" meant nothing before 2020), and the pretraining vocabulary can gradually stop matching what a live system encounters, a text analogue of the concept drift of Chapter 14's Section 14.6.2 and Chapter 18's population-stability index. Three responses are common, roughly in increasing order of cost: prompting or retrieval (Section 16.3.5) supplies current information to a frozen model; parameter-efficient fine-tuning updates a small number of added parameters on a modest domain dataset; and continued pretraining resumes the self-supervised objective on fresh text, updating the weights more thoroughly at higher cost. The choice depends on whether the problem is a knowledge gap (a missing recent fact, best fixed by retrieval) or a shift in how language is used (better fixed by continued pretraining or fine-tuning), a diagnosis that requires the ongoing monitoring Chapter 18 established for every model.

16.3.4 Large Language Models in Finance: Capabilities, Generative Modeling, and Limits

Suppose an analyst is handed a hundred-page merger filing an hour before a decision, or a portfolio manager wants a plain-English answer to a question buried in it. None of this chapter's classification or scoring tools, built to output a single label or score, can do this. Chapter 6 noted that LLMs are far more elaborate examples of the generative principle behind Naive Bayes: both learn a probability distribution, Naive Bayes over word counts given a class, an LLM over the next token given all preceding tokens, and both can be used generatively. An LLM is, mechanically, a very large stack of the self-attention layers of Section 16.3.1, trained on enormous volumes of text to predict the next token, then adapted (through instruction tuning and related techniques) to follow natural-language prompts.

In finance, this supports tasks well beyond classification and scoring: summarizing a long earnings call, extracting a table of figures from an unstructured filing, drafting a research note, or answering a question about a specific document. These tasks come with a different failure mode: hallucination, fluent, confident text that is factually wrong, such as a number invented rather than retrieved, a misattributed filing detail, or a summary that omits a covenant violation buried in a footnote.

A classifier fails with a wrong label and a computable error rate; an LLM fails with a wrong but plausible answer, harder to detect automatically and dangerous where one invented figure in a research note can be consequential. The next four sections develop the leading mitigation for hallucination (retrieval-augmented generation), how to instruct a generative model (prompting), a use of generative modeling that has nothing to do with text (synthetic data), and how an institution governs all of this.

Beyond Text: Code Generation and Multimodal Applications

Two further capabilities go beyond text-in, text-out tasks. Code generation applies next-token prediction to programming languages. An analyst can describe a calculation in plain English, for example "load this CSV of daily returns and compute the annualized Sharpe ratio and maximum drawdown," and receive working Python code, extending the pandas and NumPy analysis this book has built since Chapter 2. The code still needs scrutiny before it runs against real portfolio data: a subtly wrong annualization factor or an off-by-one window is a bug, not a hallucination in the text sense, but no less consequential.

Wulin.Suo@Queensu.ca

Multimodal models extend the architecture to inputs beyond text, reading a scanned PDF page, a chart in an investor presentation, or a table image directly, without a separate, brittle optical-character-recognition step. This matters because much financial disclosure (annual report exhibits, scanned older filings, chart-heavy investor decks) is not available as clean machine-readable text.

Reasoning Models and Long Context Windows

Two developments since 2024 have changed how these models are used. Reasoning models are trained, largely by reinforcement learning on problems whose answers can be checked automatically, such as mathematics and code, to write out a long chain of thought before answering (DeepSeek-AI 2025): the chain-of-thought prompting of Section 16.3.7, learned rather than requested. Accuracy then rises with test-time compute, the number of reasoning tokens spent per question, which turns cost and latency into a dial. Reconciling two differently defined metrics across a pair of filings justifies a long think; labeling a headline's sentiment does not. Reasoning tokens are usually billed as output tokens, the expensive side of Section 16.5.8's cost formula. Reasoning lowers error rates without removing them, and a displayed trace is not guaranteed to be the computation that produced the answer, so a reviewer should treat it as evidence, not proof.

Context windows, the number of tokens a model can read at once, have grown from a few thousand to hundreds of thousands and, as of October 2026, a million or more for several models, enough to hold an entire 10-K. This changes what retrieval is for, not whether it is needed. A question about one filing can now often be answered by reading the whole filing, which removes the risk of retrieving the wrong passage. But a firm's corpus runs to millions of documents; every token in the prompt is paid for on every query; and models use long contexts unevenly: Liu et al. (2024) found accuracy highest when the relevant passage sits near the start or end of the context and markedly lower when it sits in the middle. Retrieval also keeps citations precise and lets access control be enforced document by document (Section 16.4.2).

Reading Loan Documents in Private Credit

One extraction task shows why reading, not writing, is the hard part. Private credit, loans made directly to mostly mid-sized companies by funds and other nonbank lenders rather than syndicated by banks, had grown to about $2.1 trillion of assets and committed capital worldwide by 2023, roughly three-quarters of it in the United States, and the IMF noted signs of weaker underwriting and looser covenants as these lenders competed with banks for large deals (International Monetary Fund 2024). The loans rarely trade and are documented in bespoke agreements of a hundred pages or more, so a lender monitoring several hundred borrowers must answer the same questions from each agreement and quarterly compliance certificate: what is the maximum leverage, how is it measured, and how close is the borrower? These are the summarization and extraction tasks above, and the danger lies in the definitions. Suppose a borrower has $250 million of net debt, reported EBITDA of $40 million, and a maintenance covenant capping total net leverage at 5.50 times "Consolidated EBITDA," which the agreement defines to include add-backs, here $5 million of projected cost synergies and $3 million of restructuring charges. With every add-back, leverage is $250/48 = 5.208$ times, a pass with 5.3% of headroom, since the covenant needs only $250/5.50 = \$45.45$ million. If the definition caps add-backs at 15% of EBITDA before them, the measure is $40 + 6 = 46$ and leverage 5.435 times, still a pass but with 1.2% of headroom. On reported EBITDA alone it is $250/40 = 6.250$ times, a breach. A model that summarizes the position as "leverage of about 5.2 times against a 5.5 times covenant" has answered from the wrong clause, and has done so fluently. The safe design is the one this chapter keeps arriving at: the model extracts the definition, the cap, and each add-back as structured fields, each tied to the clause it came from (the retrieval and citation of the next subsection); a calculator computes the ratio (the tool use of Section 16.5.3); and an analyst reviews every borrower near its limit.

16.3.5 Retrieval-Augmented Generation

Retrieval-augmented generation (RAG) is the most widely deployed mitigation for hallucination in finance. Rather than relying on facts an LLM absorbed in training, possibly outdated, generic, or never seen, a RAG system first retrieves the most relevant passages from a trusted, current collection (a bank's own filings, policy manuals, or licensed research), then conditions the answer on them.

Wulin.Suo@Queensu.ca

This turns open-ended generation into something closer to open-book comprehension: the model answers from the retrieved text, and its output can be traced to a source passage, which is what an auditor or compliance reviewer needs. Retrieval applies Section 16.2.3's embedding geometry: embed the query and every candidate document in the same space, rank documents by cosine similarity to the query, and pass the top passages to the model.

Figure 16.4 lays out the pipeline in two phases. Offline, the collection is split into chunks, tagged with metadata, embedded, and indexed. Online, each query is matched against the index, candidates are re-ranked, and the best few passages go into the prompt with the question; the answer cites those passages and is checked claim by claim before anyone sees it. The worked example below walks through retrieval on a toy index, the discussion after it covers chunking, hybrid retrieval, and re-ranking, and Section 16.3.6 develops the two final checks.

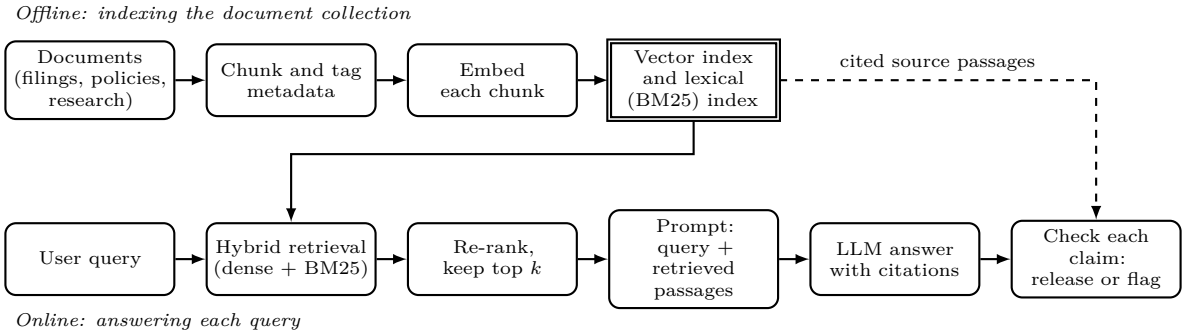


Figure 16.4: A retrieval-augmented generation pipeline. Documents are chunked, embedded, and indexed offline; each query is answered from retrieved and re-ranked passages, and the answer is checked against its cited sources before release.

Worked Example 16.11: Retrieving the Right Passage from a Toy Knowledge Base

Consider a knowledge base of four short passages about two companies, embedded in a hand-illustrative three-dimensional scheme whose dimensions loosely represent revenue, leverage, and other-company topics (Table 16.6).

Document	Passage	Embedding
Doc1	"Company X reported Q3 revenue of $420 million, up 8% year over year."	$(0.90, 0.10, 0.00)$
Doc2	"Company X's debt-to-equity ratio increased to 1.8 in Q3."	$(0.10, 0.90, 0.00)$
Doc3	"Company Y announced a new product launch in Q4."	$(0.00, 0.00, 0.90)$
Doc4	"Company X's Q3 net margin improved to 12.5% from 10.1%."	$(0.60, 0.50, 0.00)$

Table 16.6: Toy knowledge base for the retrieval-augmented generation worked example

A user asks, "What was Company X's revenue in Q3?", embedded as $(0.95, 0.05, 0.00)$.

Wulin.Suo@Queensu.ca

Computing cosine similarity between the query and each document gives

$$\cos(\text{query}, \text{Doc1}) \approx 0.9983,$$
$$\cos(\text{query}, \text{Doc4}) \approx 0.8008,$$
$$\cos(\text{query}, \text{Doc2}) \approx 0.1625,$$
$$\cos(\text{query}, \text{Doc3}) = 0.0000.$$

Doc1 is the top match by a wide margin, and only Doc1 (or, at a higher retrieval count, Doc1 and Doc4) would be passed to the model, which answers from Doc1's text: "Company X's Q3 revenue was $420 million, up 8% year over year," grounded in a verifiable source rather than invented from training data. Doc2 and Doc3, though they mention Company X or a business topic, rank far below Doc1 and Doc4, since neither answers a revenue question, the discrimination retrieval needs to make.

Chunking and Hybrid Retrieval

A production system differs from this toy in two ways. First, documents must be split into chunks before embedding, and chunk size is a trade-off: too small, and a figure is separated from the sentence saying which quarter it belongs to; too large, and one embedding averages over several topics and matches none well. Common practice is chunks of a few hundred tokens with some overlap, cut along the document's structure (sections together, tables whole) and tagged with metadata such as company, filing date, and section, so a query can be filtered to the right filing before similarity is computed. Second, dense embeddings encode what a passage is about and are weak at exact matches on the identifiers finance depends on: a ticker, a CUSIP, a clause number, "Item 1A." Hybrid retrieval therefore runs a lexical ranker, usually BM25 (Robertson and Zaragoza 2009), a refined descendant of Section 16.1.2's TF-IDF, beside the dense retriever and merges the two ranked lists (for example by reciprocal rank fusion). The top few dozen candidates then go to a re-ranker, a model that reads the query and each passage together and scores relevance more accurately than either retriever, before the best handful reach the generator.

The companion notebook's optional "at scale" section repeats this retrieval with a real sentence-embedding model over a thirty-passage knowledge base, side by side with a TF-IDF ranking, and scores Section 16.1.3's two transcripts with a transformer classifier fine-tuned on financial news. It needs the extra packages listed in the repository's `requirements-dl.txt` and downloads the pretrained model weights on its first run.

16.3.6 Evaluating a RAG System: Precision@k and a Faithfulness Score

Retrieving *a* document is not retrieving the *right* one, and a production RAG system needs to check retrieval and generation separately, not only spot-check final answers. Precision@k measures retrieval alone: what fraction of the top k retrieved documents is relevant? For the revenue question, only Doc1 answers it; Doc4 (net margin) and Doc2 (leverage) concern Company X but do not answer *this* question. Ranking by the cosine similarities already computed (Doc1, then Doc4, then Doc2, then Doc3),

$$\text{Precision@1} = \frac{1}{1} = 100\%, \qquad \text{Precision@2} = \frac{1}{2} = 50\%, \qquad \text{Precision@3} = \frac{1}{3} \approx 33.3\%,$$

since only one document in each of the top 1, 2, and 3 is relevant. This is a quantitative reason to keep k small and the similarity threshold strict: retrieving more documents mechanically dilutes precision, and every non-relevant passage passed to the model is another chance for it to draw on irrelevant context.

A second check addresses generation: given the right passage, did the answer stick to it? A faithfulness score compares the answer's embedding to the source passage's, using the same cosine geometry as retrieval. The grounded answer, "Company X's Q3 revenue was $420 million, up 8% year over year," embeds almost identically to Doc1, at $(0.89, 0.12, 0.00)$, giving

$$\text{Faithfulness} = \cos\big((0.89, 0.12, 0.00),\ \text{Doc1}\big) \approx 0.9997,$$

close to a perfect match, since the answer restates Doc1. A hallucinated answer that invents a leverage claim absent from Doc1 embeds instead at $(0.30, 0.85, 0.00)$, closer to Doc2's topic than Doc1's:

$$\text{Faithfulness} = \cos\big((0.30, 0.85, 0.00),\ \text{Doc1}\big) \approx 0.4349,$$

Wulin.Suo@Queensu.ca

far below the grounded answer. A production system can flag any answer scoring below, say, 0.85 against its source for rejection or human review. That makes the score a useful *topical-drift* check: it catches an answer that has wandered to a different subject.

It does not catch the error that matters most in finance, a wrong number on the right topic. Suppose the model answers "Company X's Q3 revenue was $240 million, down 8% year over year," transposing two digits and reversing the direction. The answer is about the same subject as Doc1, so in this topic-based embedding it sits where the grounded answer sits and scores the same 0.9997, passing the 0.85 threshold. Real embedding models behave the same way, because they encode what a sentence is about far more strongly than its digits. Even bag-of-words vectors built from the sentences (Section 16.1.2) give the wrong answer a cosine of 0.7333 against Doc1, against 0.8667 for the grounded one: lower, but nowhere near the leverage answer's 0.4349, and a threshold strict enough to reject it would also reject correct answers phrased differently.

Faithfulness therefore needs a second, *claim-level* check. Extract every verifiable claim from the answer (numbers, entities, dates, directions of change) and require each to appear in, or be computable from, the source. Here the check is mechanical: the grounded answer's three claims (420, 8, "up") are all in Doc1, while the wrong answer's (240, 8, "down") pass on only one of three, and it is rejected. Production systems go beyond exact matching by asking a natural-language-inference (entailment) model whether the source entails each claim, or a second LLM to judge each claim ("LLM-as-judge"), and frameworks such as RAGAS package claim-level faithfulness scores with retrieval metrics like precision@k. Together, the two checks give the validation practice Section 16.4.2 calls for, "sample many outputs and check them against source documents for factual grounding," a computable form.

Finance-specific benchmarks test the whole pipeline. FinanceBench (Islam et al. 2023) poses 10,231 questions about public companies' filings, each with an answer and its supporting evidence passage (150 released openly); in its original 2023 evaluation, GPT-4-Turbo with retrieval answered 81% of the sampled questions incorrectly or refused, a sobering baseline. Published scores age quickly and can leak into later models' training data, so an institution should also keep a private question set from its own documents. When an LLM grades answers at that scale, the judge's biases need controlling. Zheng et al. (2023) document three: position bias (favoring the answer shown first), verbosity bias (favoring longer answers), and self-enhancement bias (favoring its own answers). The remedies are to swap the order and count only verdicts that survive the swap, to grade against the claim list or a reference answer rather than holistically, to use a judge from a different model family, and to calibrate the judge against a human-labeled sample, the validation against ground truth Chapter 18 requires of any model.

16.3.7 Prompting Strategies for Financial Tasks

How a task is phrased to a generative model, its prompt, materially affects output quality, and three strategies recur in finance. Zero-shot prompting states the task: "Classify the sentiment of the following earnings-call excerpt as positive, negative, or neutral: *[excerpt]*." This works for common, well-defined tasks but can be inconsistent where domain conventions the model has not reliably learned apply, the finance-vocabulary issue of Section 16.1.1.

Few-shot prompting puts a few worked examples in the prompt to show the labeling convention wanted: two or three (excerpt, label) pairs, perhaps from Section 16.1.3's transcripts, followed by a new excerpt to classify. This resembles Chapter 6's supervised learning, except the "training" happens within a single prompt, with no weight updates, and the "training set" can be two or three examples.

Chain-of-thought prompting asks the model to reason step by step before answering, for example: "First identify the reported revenue figure and the prior-year figure, then compute the percentage change, then state whether this represents an acceleration or deceleration relative to the previous quarter's growth rate, showing each step." For multi-step numerical tasks, such as computing a ratio from figures buried in a paragraph or reconciling two differently defined metrics, this measurably reduces errors compared with asking for the number directly, because it puts the intermediate steps in the output where they can be checked, much as this book shows every intermediate step of a calculation, a practice established in Chapter 7.

16.4 Generative AI in Practice

Generation raises problems consumption does not, from synthetic data to governance to domain adaptation.

Wulin.Suo@Queensu.ca

16.4.1 Generative Models for Synthetic Financial Data

A different application of generative modeling has nothing to do with text: generating synthetic numerical data (stress-test scenarios, order-book sequences, extra examples of a rare event class) that resembles real data closely enough to train or test other models. Two architectures dominate.

A generative adversarial network (GAN) pits two neural networks against each other: a generator G maps random noise z, drawn from a simple fixed distribution p_z such as a standard normal, to synthetic samples $G(z)$, and a discriminator D tries to distinguish real samples x, drawn from the data distribution p_{data} being imitated, from generated ones. They are trained jointly via

$$\min_G \max_D \; \mathbb{E}_{x\sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z\sim p_z}\left[\log\left(1 - D(G(z))\right)\right]. \tag{16.8}$$

Each term rewards D for half its job: the first for assigning high probability to real samples, the second for assigning low probability to fakes. Only the second involves G, so the generator's objective is to drive $D(G(z))$ up while the discriminator pushes it down. As training proceeds, each improves against the other, and at convergence the discriminator can no longer reliably tell generated samples from real ones. In finance, GANs have been used to augment a fraud model's minority class, extending Chapter 15's resampling techniques such as SMOTE with synthetic rather than interpolated examples, and to generate limit-order-book sequences for backtesting execution algorithms without exposing client order flow.

The companion notebook evaluates this minimax game on Chapter 15's fraud-scoring data: a fixed, illustrative discriminator $D(x) = \sigma(2.0x - 5.0)$ assigns transaction 1's fraud amount, standardized to 3.1, a probability of being real of $D(x_{\text{real}}) = 0.7685$. (Chapter 15 names this standardized feature with the conventional z of a standard score; here z is the generator's noise input, so the feature is called by its role.) An early-training generator, still producing samples near a typical legitimate transaction's scale, might output $G(z) = 1.0$, which the discriminator flags as almost certainly fake, $D(G(z)) = 0.0474$, a gap of 0.7211 from $D(x_{\text{real}})$.

A later generator producing a fraud-scale sample, $G(z) = 2.9$, gets $D(G(z)) = 0.6900$, closing the gap to 0.0786: the generator's loss, $-\log D(G(z))$, falls from 3.0486 to 0.3711, a numerical measure of how much harder it has become to catch.

GANs are also notoriously hard to train. Because generator and discriminator are optimized against each other rather than toward one shared objective, training can oscillate instead of converging, or fall into mode collapse: the generator finds a few samples that reliably fool the discriminator and stops exploring the rest of the distribution, producing fraud examples or stress scenarios that are realistic but too uniform to augment a thin dataset. This instability, which not every generative architecture shares, is one reason diffusion models, described next, have become the more common choice for new synthetic-data applications despite being costlier to sample from.

Diffusion models take a different approach, motivated by physical diffusion: a forward process gradually adds noise to real data over many steps until it is indistinguishable from pure noise, $x_t = \sqrt{\alpha_t}\, x_0 + \sqrt{1-\alpha_t}\, \varepsilon$, where x_0 is the original data point, ε is standard-normal noise, and α_t decreases from near 1 (step 0, almost no noise) toward 0 (fully noised) as t increases. A neural network is trained to reverse this, removing noise one step at a time, so that running the learned reverse process from pure noise produces a new sample resembling the original distribution.

Worked Example 16.12: One Forward Diffusion Step

Consider a toy stress-test scenario variable, a multiplier applied to a baseline loss, with baseline (undiffused) value $x_0 = 1.00$ (no stress) and a noise draw $\varepsilon = -0.50$. At an early diffusion step with $\alpha_t = 0.80$,

$$x_t = \sqrt{0.80}\,(1.00) + \sqrt{0.20}\,(-0.50) = 0.6708.$$

At a later, noisier step with $\alpha_t = 0.55$,

$$x_t = \sqrt{0.55}\,(1.00) + \sqrt{0.45}\,(-0.50) = 0.4062.$$

As α_t falls from 0.80 to 0.55, the scenario value moves from 0.6708 down to 0.4062, drifting from the baseline of 1.00 toward the noise draw of -0.50; continuing to $\alpha_t \to 0$ would carry x_t to ε itself, pure noise with no

Wulin.Suo@Queensu.ca

trace of the original scenario. The two coefficients are square roots chosen so that $\left(\sqrt{\alpha_t}\right)^2 + \left(\sqrt{1-\alpha_t}\right)^2 = 1$: they hold the *variance* of x_t constant as noise replaces signal, and they are not a weighted average of x_0 and ε, since they sum to more than one. A same-signed noise draw can therefore push x_t temporarily outside the range spanned by x_0 and ε before it settles toward ε. A trained diffusion model learns to run this in reverse, starting from noise and reconstructing a plausible, never-observed stress scenario that Chapter 8's stress testing could use to probe losses beyond anything in the historical sample.

16.4.2 Governance and Risk Management for Generative AI

Chapter 15 developed a three-lines-of-defense governance framework for fraud and compliance models; generative AI needs the same discipline, applied to risks deterministic models did not raise. Three deserve attention.

First, non-determinism: the same prompt submitted twice can produce two outputs, which breaks validation approaches that assume a fixed input gives a reproducible output, and pushes validation toward sampling many outputs. Second, prompt injection: adversarial text in a retrieved document, an uploaded file, or ordinary user input can try to hijack the system, for example an instruction hidden in a retrieved filing telling the model to ignore its task, an attack surface with no counterpart in earlier chapters' classifiers. Third, data leakage: a RAG system with access to sensitive internal documents can, if not carefully scoped, surface information to a user who should not see it, an inadvertent access-control failure.

These risks change how the third line's independent validation, introduced in Chapter 15, operates. Rather than checking predictions against a fixed labeled test set, it must sample many outputs, check them against source documents for factual grounding (the check Section 16.3.5's retrieval supports), and test for prompt injection and data leakage, before deployment and on an ongoing basis as the model, document collection, or user population changes. Regulators have generally applied existing model risk management expectations rather than writing new rules, although the U.S. interagency guidance that replaced SR 11-7 in April 2026, SR 26-2, explicitly leaves generative and agentic AI outside its scope pending separate treatment (Federal Reserve, OCC, and FDIC 2026). In practice a generative AI deployment is still expected to meet the same expectations, documented validation, ongoing monitoring, and a clear escalation path, though the tools for meeting them differ.

16.5 Advanced Topics: Fine-Tuning, Agents, and Domain Adaptation

The base LLM of Section 16.3.4, trained only to predict the next token, is not yet the instruction-following assistant of Section 16.3.5 and this chapter's case vignette. Four further ideas close that gap.

16.5.1 Fine-Tuning and Reinforcement Learning from Human Feedback

A model pretrained only to predict the next token is an unreliable assistant: asked a question, it may as easily continue writing in the same style as answer it, since nothing in next-token prediction teaches it to follow an instruction or to prefer a helpful, accurate response over a fluent but unhelpful one. After pretraining, a model is typically trained in two further stages: supervised fine-tuning on curated (prompt, ideal response) pairs, teaching it to follow instructions, and reinforcement learning from human feedback (RLHF), which uses human preference judgments between pairs of candidate outputs to train a reward model, then adjusts the LLM to produce outputs that reward model scores highly. The reward model itself is typically trained as a preference classifier: given two candidate outputs with reward scores r_A and r_B, the Bradley-Terry model estimates the probability that a human would prefer output A as

$$\Pr(A \succ B) = \text{sigmoid}(r_A - r_B) = \frac{1}{1 + e^{-(r_A - r_B)}}. \tag{16.9}$$

Worked Example 16.13: Reward-Model Preference Probability

Suppose a reward model scores a source-accurate earnings-call summary at $r_A = 2.1$ and a fluent but subtly inaccurate one at $r_B = 0.8$. The preference probability is $\text{sigmoid}(2.1 - 0.8) = \text{sigmoid}(1.3) \approx 0.7858$: the

Wulin.Suo@Queensu.ca

reward model favors the accurate summary about 78.6% of the time, and RLHF nudges the LLM's outputs toward what the reward model scores highly, in aggregate across many such comparisons.

16.5.2 Parameter-Efficient Fine-Tuning: Low-Rank Adaptation (LoRA)

RLHF and supervised fine-tuning describe *what* a model is trained to do after pretraining; a practical question is *how*. Updating every weight, full fine-tuning, requires storing a second full copy of the parameters (the update) beside the original, and repeating this for every task or client is prohibitively expensive at the scale of a modern LLM.

Low-Rank Adaptation (LoRA) is the most widely used solution: instead of learning a dense update ΔW for a $d \times d$ weight matrix, it constrains the update to a low-rank decomposition, $\Delta W = BA$, where B is $d \times r$, A is $r \times d$, and the rank r is far smaller than d. Only B and A are trained; W is frozen, and the adapted forward pass uses $W + BA$ in place of W.

Worked Example 16.14: Parameter Reduction from Low-Rank Adaptation

Consider a single $d = 4{,}096$ weight matrix, a realistic dimension for one projection inside a modern LLM. Full fine-tuning of it requires learning all $d^2 = 4{,}096^2 = 16{,}777{,}216$ entries of ΔW. Using LoRA with rank $r = 8$, the trainable parameters are instead only those of B ($d \times r$) and A ($r \times d$),

$$2 \times d \times r = 2 \times 4{,}096 \times 8 = 65{,}536,$$

a reduction factor of $16{,}777{,}216/65{,}536 = 256\times$ for this matrix, with a comparable reduction for every matrix LoRA is attached to.

This is why LoRA has become the default way to adapt a large model to a bank's internal document style or an asset class's terminology. It avoids both the storage cost of a duplicate model per use case and the compute cost of updating billions of parameters. The frozen base model, possibly quantized as described below, can be shared across many clients or tasks, with only a small task-specific B and A pair, tens of thousands of parameters rather than billions, swapped in per use case.

16.5.3 LLM Agents and Tool Use

Computing and cross-checking a company's P/E ratio requires looking up its price, pulling its latest earnings from a filing, dividing, and comparing against a sector benchmark, a sequence no single LLM call, which produces text in one pass, can carry out. An agent extends a single LLM call into a multi-step loop: the model can call an external tool (a retrieval system, a calculator, a database query, or a trading API), observe the result, and choose a next action, repeating until it produces a final answer. This composes Section 16.3.5's retrieval with ordinary computation.

In practice the loop runs on function calling, also called tool calling. The developer describes each tool by a name, a plain-language description, and a JSON schema for its arguments; when the model wants a tool, it emits a structured call, in effect `calculator(57.87/3.15)`, and the application checks the call against the schema, runs it, and returns the result as a new message. The same machinery provides structured outputs: constraining a model to produce JSON conforming to a schema, which turns "extract the figures from this filing" into typed fields a downstream system can validate rather than prose it must parse. The Model Context Protocol (MCP), an open standard introduced by Anthropic in November 2024 and transferred in December 2025 to the Agentic AI Foundation, a Linux Foundation project (Anthropic 2024; Linux Foundation 2025), standardizes how tools and data sources are offered to models, so that one connector to a market-data service or a document store can serve any compliant application.

Worked Example 16.15: A Three-Step Agent Computing and Cross-Checking a P/E Ratio

Suppose an agent is asked, "What is Company X's price-to-earnings ratio using its Q3 diluted EPS and quarter-end share price?" The agent first calls a retrieval tool (Section 16.3.5's mechanism), retrieving two facts: diluted EPS of \$3.15 and a quarter-end closing price of \$57.87. It then calls a calculator tool rather than doing the arithmetic itself, computing $\text{P/E} = 57.87/3.15 \approx 18.37$. This removes a whole category of

Wulin.Suo@Queensu.ca

arithmetic-hallucination risk: a calculator's division is exact by construction, while an LLM can make an arithmetic error despite retrieving correct inputs.

A more robust agent takes a third step, calling an independent tool to cross-check the result before presenting it. Suppose it queries a market-data vendor's reported P/E for Company X and retrieves 19.10. Compared with its own value,

$$\text{Discrepancy} = \frac{19.10 - 18.37}{18.37} \approx 3.97\%,$$

exceeds a pre-set tolerance of, say, 2%, so rather than presenting either number as final, the agent flags the disagreement, reporting both figures with a note that the gap may reflect a definitional or timing difference (the vendor may use trailing rather than diluted EPS, or a different price snapshot) worth investigating before either is used in a client-facing report.

The cross-check costs one tool call but turns a possibly wrong number into either a validated one or a flagged discrepancy, the independent-verification discipline Chapter 8's model-validation framework requires of a classical risk model.

An agent that can act also inherits Section 16.4.2's risks in sharper form, because a successful prompt injection now triggers tool calls rather than bad text. Indirect prompt injection, instructions planted in a web page, an email, or a retrieved filing that the agent reads as data, was demonstrated against deployed applications by Greshake et al. (2023), and prompt injection heads the OWASP Foundation's 2025 list of risks for LLM applications, with "excessive agency" also on it (OWASP Foundation 2024). No model-level defense is yet reliable, so the controls are architectural: least-privilege permissions (a research agent gets read-only data tools, and no trading or payment tool); every tool result and retrieved document treated as untrusted data, never as instructions; human approval before any irreversible or external action, such as sending an email, placing an order, or moving money; hard limits on spending and on the number of calls; and a complete log of every tool call for audit, the agent-level counterpart of Chapter 15's three lines of defense.

16.5.4 Agentic AI in Finance: Planning, Memory, Evaluation, and Failure

The P/E agent took three steps, each checkable by eye. The systems banks began piloting in 2025 and 2026 take tens of steps across several internal systems (refreshing a client's know-your-customer file, reconciling a failed trade, assembling a credit memo), and at that length the questions change. How does the agent decide what to do next? What may it touch? What does it remember between steps and tasks? How is it evaluated, when the output is a sequence of actions? And how does it fail? This subsection takes each in turn; its worked example shows why the last dominates: reliability that looks excellent per step becomes mediocre per task.

The reason-act-observe loop. Most agents run some version of the ReAct pattern of Yao et al. (2023), shown in Figure 16.5. At each step the model writes a short reasoning trace (what it knows, what it still needs), emits one action, a structured tool call, and receives the tool's output as an observation appended to its context. The loop ends when the model gives a final answer, escalates to a human, or exhausts a step or spending budget. Two planning styles sit on top of this loop. In an interleaved style the agent decides one step at a time, which adapts to surprises but can wander; in a plan-then-execute style it first writes the whole plan ("retrieve the client's ownership records, screen each beneficial owner against the sanctions list, compare against the file on record, draft the exceptions"), executes it, and replans only when a step fails, which a reviewer can read in advance and check afterward. Regulated workflows tend to prefer the second for that reason.

Permissions and sandboxing. The permission check in Figure 16.5 is where least privilege becomes engineering. Tools are sorted by consequence: read-only tools (a filing search, a market-data lookup) run freely; write tools whose effects can be undone (drafting a document, opening a ticket) run with logging; and tools whose effects cannot be undone or leave the firm (messaging a client, releasing a payment, submitting an order) run only after a named human approves that specific call, with its arguments, not the task in general. Code the agent writes runs in a sandbox, an isolated container with no network access or credentials, so a wrong or injected script damages nothing outside it. And the agent acts under its own service identity, never a borrowed human login, so every action is attributable and its entitlements reviewable like anyone

Wulin.Suo@Queensu.ca

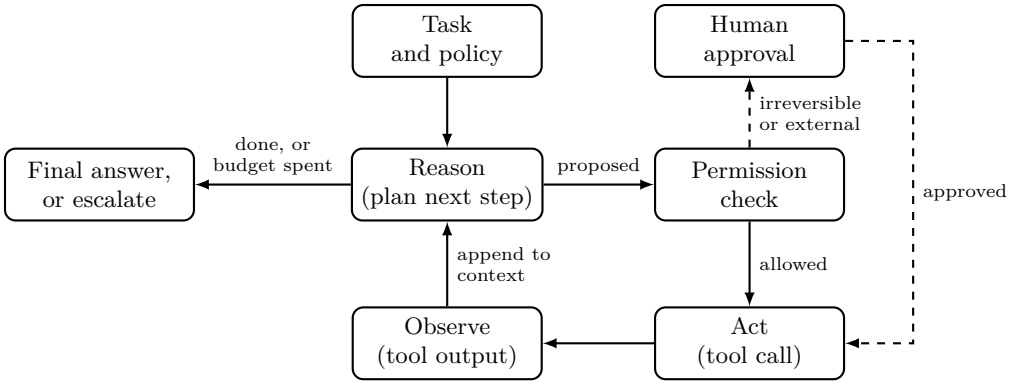


Figure 16.5: The reason-act-observe loop of an LLM agent, with a permission check between the model's proposed action and its execution, and human approval for irreversible or external actions

else's; BNY, for example, gives each of its more than one hundred agent "digital employees" its own login credentials (Axios 2025).

Memory. An agent has two kinds. Short-term memory is the context window, the running transcript of thoughts, actions, and observations, which is finite and degrades before it is full: models use information at the start and end of a long context more reliably than in the middle (Liu et al. 2024), so long tasks summarize old steps. Long-term memory is an external store, typically a vector database searched with Section 16.3.5's retrieval, holding past interactions, client preferences, or lessons from earlier tasks. That is where finance-specific risks collect. A planted instruction written into memory persists into every later task that retrieves it, turning a one-time prompt injection into a standing one; a stored fact goes stale (a client's address, a counterparty's rating) and is retrieved with the same confidence as a fresh one; and memory shared across clients or desks can carry material nonpublic information across the information barriers a bank must maintain. Memory therefore needs the same retention schedule, access control, and periodic review as any other client record.

Evaluating agents. An agent is evaluated on a suite of realistic tasks, with two kinds of measurement. The first is outcome: did the task end in the correct state? The τ-bench benchmark (Yao et al. 2024), for instance, compares the database at the end of each simulated customer-service conversation with the annotated goal state, rather than trusting the agent's report, and asks whether the agent succeeds on all of k repeated attempts, its pass^k reliability; when it was published in June 2024, even the strongest function-calling agents solved fewer than half the tasks, and their pass^8 in the retail domain was below 25%. The second is trajectory review, reading the steps themselves, because a correct end state reached by a wrong path (a forbidden data source consulted, an approval skipped, a calculation done by the model instead of the calculator) is a failure outcome scoring cannot see. In practice a firm reports task success rate, the rate and appropriateness of escalations, cost and steps per task, and the findings of humans reviewing a sample of trajectories, the agent-level version of Chapter 6's held-out evaluation.

Worked Example 16.16: How Per-Step Reliability Compounds

Suppose a task requires k steps, each completed correctly with probability p, and, as a first approximation, that errors at different steps are independent. The task succeeds only if every step does:

$$\Pr(\text{task succeeds}) = p^k. \tag{16.10}$$

Table 16.7 evaluates this for two per-step reliabilities. An agent that is right 95% of the time at each step completes a ten-step task correctly only $0.95^{10} = 59.87\%$ of the time, and a twenty-step task 35.85% of the time. Even at 99% per step, one task in five fails at twenty steps. Inverting the equation shows how demanding a task-level target is: a 95% task success rate requires $p = 0.95^{1/k}$ per step, which is 0.98979 at five steps, 0.99488 at ten, and 0.99744 at twenty.

Wulin.Suo@Queensu.ca

Per-step reliability p	$k = 5$	$k = 10$	$k = 20$
0.95	0.7738	0.5987	0.3585
0.99	0.9510	0.9044	0.8179

Table 16.7: Probability that a k-step agent task succeeds, p^k, for independent steps

Now add a verification step at the end of each task, an independent check of the kind the P/E agent ran, which catches a fraction c of the tasks that went wrong and also, occasionally, flags a task that went right, at a false-alarm rate f. Every flag goes to a human reviewer. Over N tasks the expected review load R and the expected number of silent failures S, wrong outputs that pass the verifier unflagged, are

$$R = N\left[c\,(1-p^k) + f\,p^k\right], \qquad S = N\,(1-c)\,(1-p^k). \tag{16.11}$$

Take $N = 1{,}000$ ten-step tasks, a verifier that catches $c = 80\%$ of failures, and a false-alarm rate of $f = 2\%$. At $p = 0.95$, 401.3 of the tasks contain an error; the verifier flags $0.80 \times 401.3 = 321.0$ of them and $0.02 \times 598.7 = 12.0$ good tasks besides, so humans review 333.0 tasks, a third of the volume, and $0.20 \times 401.3 = 80.3$ wrong outputs still go through unflagged. At $p = 0.99$, only 95.6 tasks fail; humans review $76.5 + 18.1 = 94.6$ tasks, under a tenth of the volume, and 19.1 errors go through silently. Without a verifier, all the failures, 401.3 or 95.6, would be silent.

Three lessons follow. First, per-step reliability is the economic lever: moving from 95% to 99% per step cuts the review load by a factor of about 3.5 and silent failures by about 4.2, which decides whether an agent saves analyst time or merely moves it into review. Second, a verifier converts silent failures into review work but never removes them; the residual S is what the firm accepts, and c must be estimated on tasks with known errors, not assumed. Third, as the agent improves, more of what the verifier flags is false alarm: the flags' precision, $321.0/333.0 = 96.4\%$ at $p = 0.95$, falls to $76.5/94.6 = 80.9\%$ at $p = 0.99$, because the same 2% false-alarm rate applies to many more good tasks, the base-rate effect of Chapter 1's Bayes' theorem example. Independence is the assumption to doubt. Errors from one shared cause, a misread instruction or a stale fact in memory, cluster in the same tasks, which leaves more tasks clean than p^k predicts but makes those errors systematic; and a verifier built on the agent's own model likely shares its blind spots, so its true c on those errors is below its average.

Failure modes. The worked example is the arithmetic of the most basic failure, small errors compounding over many steps. The others are behavioral. An agent can loop, retrying a failing call or re-planning without progress until a budget stops it, which is why step and spending limits are design requirements. It can call a tool with a hallucinated argument, a plausible but wrong account number or ticker, which a schema check catches only if the value is malformed. It can pursue a proxy for the goal, closing reconciliation breaks by booking offsetting entries rather than finding their causes, the reward hacking Chapter 19 warns about. It can report success on a task it only partly completed, which is why evaluation checks the end state, not the agent's account. It can escalate too often to be useful or too rarely to be safe. And it remains exposed to prompt injection, now with memory to make it persistent. Industry forecasters expected many projects to fail on these grounds: in June 2025 Gartner predicted that over 40% of agentic AI projects would be canceled by the end of 2027, citing escalating costs, unclear business value, and inadequate risk controls (Gartner 2025).

What banks are doing. As of October 2026, the public record shows banks moving from pilots to bounded deployments with three common features: the work is internal, middle- and back-office rather than client-facing; the agent drafts or prepares rather than decides; and a named human remains accountable. Citi began piloting agentic capabilities in its in-house Stylus Workspaces platform for 5,000 employees in September 2025, letting one prompt run a multi-step task such as researching a client and compiling a profile from public and internal sources (Torres 2025). BNY described more than one hundred agent "digital employees" working alongside staff by October 2025, with tasks such as validating payment instructions (Axios 2025). In February 2026 Goldman Sachs said it had spent six months working with embedded Anthropic engineers to build agents for trade and transaction accounting and for client vetting and onboarding, the know-your-customer work Chapter 15 described, and that the effort was still at an early stage (CNBC 2026). In August 2026 DBS extended an agentic corporate-credit tool from a 150-person pilot to about 1,500 bankers:

Wulin.Suo@Queensu.ca

specialized agents handle more than 70 tasks to produce a review-ready first draft of a credit memo from annual reports, industry research, and internal records, while relationship managers and credit risk managers remain responsible for the final memo; the bank's stated aim is to cut by at least 30% the time spent on credit memos and related work, which it says can take up to 40% of a relationship manager's time (DBS 2026). These are the firms' own descriptions and targets, not independently measured results. Each applies this subsection's arithmetic: a credit memo assembled from dozens of retrieval and drafting steps is a long-k task, and the human sign-off is the verifier. How supervisors expect such systems to be governed is the subject of Chapter 18.

16.5.5 Mixture-of-Experts: Scaling Without Proportional Compute

Making a language model more capable usually means making it larger, but a larger dense feedforward network makes every token more expensive, whether or not its context needed the full capacity. A mixture-of-experts (MoE) architecture replaces the large feedforward network in each transformer layer with several smaller "expert" networks and a router that sends each token to a handful of them, so capacity grows without a proportional increase in compute per token.

Worked Example 16.17: Sparse Activation Fraction

Consider a MoE layer with 8 experts of 6 billion parameters each (48 billion total expert parameters) that routes each token to its top 2 experts. Each token activates $2 \times 6 = 12$ billion of the 48 billion expert parameters, an active fraction of $12/48 = 0.25$, or 25%: the capacity of a 48-billion-parameter layer at the per-token cost of a 12-billion-parameter one.

16.5.6 Domain-Adapted Financial Language Models: BloombergGPT

Section 16.1.3's Loughran-McDonald dictionary made a lexicon-based version of a broader point: tools built on general-purpose text underperform on financial language. BloombergGPT, announced by Bloomberg in 2023, applies the principle to a full-scale LLM: a 50-billion-parameter model trained from scratch on a mixed corpus of over 700 billion tokens, roughly 363 billion of Bloomberg's own financial data (news, filings, and other proprietary text spanning decades) and roughly 345 billion of general-purpose text, with 569 billion tokens sampled for training. It outperformed similarly sized general models on financial benchmarks (sentiment analysis, financial named-entity recognition, and financial question answering) while remaining competitive on general benchmarks, confirming at LLM scale the domain-specificity argument of Section 16.1.1.

The later evidence is more mixed. Within months, Li et al. (2023) found that general-purpose GPT-4, with no finance-specific pretraining, could outperform BloombergGPT on financial text tasks, although fine-tuned smaller models still beat it at structured extraction such as named-entity and relation recognition. Practice has broadly followed: as of October 2026, a general frontier model with retrieval over the institution's own documents is a common starting point, and domain-specific training tends to earn its cost for narrow, high-volume tasks, for data that cannot leave the firm, or for small models that must run in-house. The domain-specificity argument survives; what has changed is where domain knowledge is most cheaply supplied, at retrieval and fine-tuning time rather than by pretraining from scratch.

16.5.7 Quantization: Deploying a Large Model at Lower Cost

An institution that wants to run a model like BloombergGPT on its own infrastructure rather than send every query to an external API, a real consideration given Section 16.4.2's data-leakage concern, faces a direct memory and compute cost. Quantization reduces it by storing each parameter with fewer bits than were used in training, trading a little accuracy for a large reduction in memory.

Worked Example 16.18: Memory Footprint Under Quantization

A 50-billion-parameter model held at 16-bit precision (2 bytes per parameter) requires $50 \times 10^9 \times 2 = 100 \times 10^9$ bytes, 100 gigabytes, just to store its weights, before activations or a long conversation's key-value cache. Quantizing to 8-bit precision (1 byte per parameter) halves this to 50 gigabytes, and 4-bit precision (0.5 bytes

Wulin.Suo@Queensu.ca

per parameter) brings it to 25 gigabytes, a fourfold reduction from 16-bit. This trade-off, some accuracy lost for a model that fits on cheaper hardware and answers faster, often decides whether an institution can run a domain-adapted model in-house at all.

16.5.8 Worked Example 16.19: API Cost versus Self-Hosting at Scale

Quantization's payoff is easiest to see by pricing the alternative, framed as a break-even volume because API prices move far faster than this book's print cycle. The two cost structures differ in kind. An API bills per token, so its cost is purely variable, and output tokens cost more than input tokens, typically four to six times as much, since generating a token takes more compute than reading one. Self-hosting is mostly a fixed cost: suppose the 4-bit-quantized 50-billion-parameter model runs on one dedicated cloud GPU node reserved around the clock at $20 an hour, or $480 a day ($14,400 a month), whether it is busy or idle, with capacity for 4,000 queries an hour, 96,000 a day.

Let a query carry n_{in} input tokens (the prompt plus any retrieved context) and n_{out} output tokens, with API prices p_{in} and p_{out} per million tokens. Self-hosting wins once daily query volume Q exceeds

$$Q^{*} = \frac{\$480}{(n_{\text{in}}\, p_{\text{in}} + n_{\text{out}}\, p_{\text{out}})/10^{6}}.$$

Take an institution's financial-research assistant handling 40,000 queries a day at 800 input and 200 output tokens each. The prices are the parameter that dates. At the 2023 list price of the original GPT-4, $30 per million input tokens and $60 per million output tokens, a query cost $0.036, the API bill came to $1,440 a day, three times the self-hosted node, and the break-even was only 13,333 queries a day. As of October 2026, published list prices for mainstream models span roughly $0.05 to $10 per million input tokens and $0.50 to $50 per million output tokens, from the smallest models to the frontier ones, with premium reasoning tiers priced higher still. At an illustrative frontier-tier price of $5 in and $25 out, a query costs $0.009, the same workload costs $360 a day through the API, cheaper than the $480 node, and self-hosting wins only above 53,333 queries a day. At an illustrative mid-tier price of $2 in and $10 out, the break-even rises to 133,333 queries a day, more than the node can serve, and at a small model's $0.10 in and $0.50 out, about $7 a day buys the whole workload. Input and output asymmetry matters as much as the headline rate: tilt the same 1,000 tokens toward long generated answers, 200 in and 800 out, and the frontier-tier cost per query rises to $0.021, cutting the break-even to 22,857 queries a day.

What survives price changes is the structure, not any single number. Falling per-token prices have pushed the break-even volume up roughly fourfold or more since 2023, so a pure cost case for self-hosting now needs very high volume, an output-heavy workload, or a premium model with a high API price. The memory-footprint calculation still makes self-hosting feasible at all: a model that fits on one GPU only at 4-bit precision keeps the fixed cost at one node rather than a multi-GPU cluster. And cost is rarely the whole decision: Section 16.4.2's data-leakage concern can favor self-hosting even below the break-even, because every query stays inside the institution's infrastructure.

16.6 Case Vignette: AI @ Morgan Stanley

In September 2023, Morgan Stanley announced that AI @ Morgan Stanley Assistant, built together with OpenAI on GPT-4, was fully live for its wealth management financial advisors and support staff. In this chapter's terms the assistant is a retrieval-augmented generation system: it gives advisors natural-language access to the firm's "intellectual capital," roughly 100,000 research reports and internal documents, retrieving the passages relevant to a question and generating an answer grounded in them, the mechanism Section 16.3.5 worked through on four toy documents. Morgan Stanley later reported over 98% adoption among advisor teams, and that the share of research content advisors could effectively find and use rose from roughly 20% to roughly 80%, a direct measure of the retrieval problem at large scale.

The firm then applied the same models to a structurally different task: AI @ Morgan Stanley Debrief, which uses OpenAI's Whisper speech-to-text model with GPT-4 to turn a client meeting recording, made only with the client's consent, into a structured summary, client notes filed automatically into the customer relationship management system, and a draft follow-up email an advisor reviews and sends. This is the summarization capability of Section 16.3.4, deployed on sensitive input (a recorded client conversation) rather

Wulin.Suo@Queensu.ca

than a public filing, which is why consent, access control, and human review before anything reaches a client are first-class design requirements, the discipline Section 16.4.2 argued a deployment needs from the outset.

A third tool, AskResearchGPT, extended the same retrieval-augmented pattern into the firm's institutional research business: a RAG system validated for one internal use tends to generalize to others faster than building anew. The case is instructive because its two headline results, near-total adoption and a fourfold increase in effective document accessibility, came not from a more powerful model than other firms could license, but from the unglamorous work this chapter has emphasized: a well-curated retrieval corpus, retrieval grounded in the cosine-similarity geometry of Section 16.2.3, and a governance structure (consent for Debrief's recordings, human review of drafted outputs) that made deployment at scale defensible.

16.7 Challenges in Natural Language Processing in Finance

Several challenges recur across the techniques this chapter developed, and many connect to themes from earlier chapters.

Domain-specific vocabulary defeats generic tools. Lexicons and language models trained on general text misread financial language (Section 16.1.1), which is why finance-specific resources like the Loughran-McDonald dictionary exist and why even a large general model benefits from finance-specific fine-tuning or careful prompting.

A weak signal is still noisy, not deterministic. Section 16.2.1's signal beat buy-and-hold on mean return and Sharpe-like ratio while still being wrong on two of eight events: an NLP-derived signal is a probabilistic input to a strategy, not a certainty, as Chapter 6 emphasized about model evaluation.

Hallucination is a different failure mode from misclassification. A classifier's error rate is computable from a confusion matrix (Section 16.2.2); an LLM's fluent, wrong answers (Section 16.3.4) have no equally clean error rate, since detecting a hallucination often requires the expertise the tool was meant to save. Retrieval-augmented generation (Section 16.3.5) mitigates this without eliminating it: a model can still misread or misquote a correctly retrieved passage.

Generative AI introduces risks classical models never raised. Non-deterministic outputs, prompt injection, and data leakage (Section 16.4.2) have no counterpart in the deterministic models of Chapters 6 through 15, and require validation by statistical sampling of outputs rather than checking a single fixed prediction.

Explainability and audit requirements constrain deployment. As Chapter 18 develops, a trading decision or disclosure classification influenced by an NLP model may need to be explained after the fact, which is harder for the attention-based architectures of Section 16.3.1 than for the inspectable probabilities of Section 16.2.2's Naive Bayes classifier.

Data licensing and evaluation are harder than they look. High-quality financial text, especially real-time news and licensed feeds, is expensive and contractually restricted, and evaluating an NLP system is harder than evaluating a numerical model, since two humans often disagree about the "correct" sentiment or summary of a passage, so even the ground truth is somewhat subjective.

16.8 Key Takeaways

Financial text must be converted into numerical features before this book's earlier tools apply: bag-of-words and TF-IDF (Section 16.1.2) give sparse count vectors, the Loughran-McDonald lexicon (Section 16.1.3) gives a finance-specific sentiment score, and word embeddings (Section 16.2.3) give dense vectors that capture semantic similarity geometrically.

A lexicon-based sentiment score is a useful but noisy trading signal: Section 16.2.1's 75% hit rate still beat buy-and-hold on average return and consistency, and generating the signal is only half the task Chapter 13 develops. It should be judged by an event study, on abnormal rather than raw returns and pooled over many events, since the market explained more than half of one event's reaction and a single event's cumulative abnormal return was indistinguishable from noise. Naive Bayes applies directly to document classification (Section 16.2.2), including ambiguous cases where its independence assumption shows most.

Self-attention (Section 16.3.1) lets transformer models, including the LLMs of Section 16.3.4, build context-dependent representations of text, and their generative capability supports summarization and

Wulin.Suo@Queensu.ca

extraction well beyond classification, at the cost of a new failure mode, hallucination, that a computable error rate cannot fully capture. Retrieval-augmented generation (Section 16.3.5) is the leading mitigation, grounding answers in verifiable source passages, while prompting strategy (Section 16.3.7), zero-shot, few-shot, or chain-of-thought, shapes how reliably a model performs a task without changing the model.

Generative modeling extends beyond text: GANs and diffusion models (Section 16.4.1) generate synthetic numerical data, minority-class fraud examples or stress scenarios beyond the historical sample, by fooling an adversarial discriminator or reversing a noising process. All of this requires governance (Section 16.4.2) that extends Chapter 15's three lines of defense to risks, non-determinism, prompt injection, data leakage, that deterministic models never presented.

In Section 16.5, RLHF's reward modeling turns human preferences into a trainable signal, and agents compose retrieval with computation through function calling (delegating arithmetic to a calculator tool), with least-privilege permissions and human approval as safety controls. Over a long task, reliability compounds as p^k (Section 16.5.4), so an agent right 95% of the time per step completes a ten-step task only about 60% of the time, and per-step reliability, not headline capability, sets how much human review an agent saves or creates. Mixture-of-experts architectures decouple model capacity from per-token compute, and BloombergGPT carries the domain-specificity argument to LLM scale, although general frontier models with retrieval have since often matched it. Finally, readability metrics like the Gunning Fog index (Section 16.1.4) turn a regulatory concern, whether disclosure is understandable, into an automatable measurement.

16.9 Suggested Exercises

16.1 Using Table 16.2's method, compute the idf and tf-idf weight (in D_2) for a seventh term, "confident," which appears once in D_3 only, and explain why its idf is the largest of any term considered so far.

16.2 Using Section 16.1.3's method and word lists, compute the tone score for the sentence "results were weak and management remains cautious about the coming quarter despite improved margins."

16.3 Using Table 16.3's method, add a ninth event with tone score $+0.11$ and next-day return -0.6%, and recompute the strategy's hit rate, mean return, and Sharpe-like ratio.

16.4 Explain, using events 4 and 7 in Table 16.3, why a tone score close to zero is a weaker signal than one far from zero, and propose a modification to the trading rule that would account for this (for example, only trading when |tone| exceeds some threshold).

16.5 Using Section 16.2.2's fitted word probabilities, classify a new sentence with word counts (default: 0, borrower: 1, counterparty: 1, volatility: 0, rate: 1, currency: 0), showing the unnormalized class scores and the resulting posterior probabilities.

16.6 Using Table 16.5, explain in words why this classifier's precision is perfect (1.00) while its recall is not, and describe a business setting in which that trade-off (never a false alarm, but occasionally missing a true case) would be preferable to the reverse.

16.7 Using Section 16.2.3's toy embeddings, add a seventh term, "dividend" $= (0.70, 0.40)$, compute its cosine similarity to "stock" and to "bond," and explain which existing term it should be geometrically closest to and why.

16.8 Using Section 16.2.3's toy embeddings, compute bearish $-$ bullish $+$ stock, and explain what analogy this attempts to capture, and why, unlike the stock $-$ equity $+$ bond example, the result does not land exactly on any of the chapter's six labeled terms.

16.9 Using Section 16.3.1's worked example, recompute the self-attention output for the token "rallied" (using query $Q_{\text{rallied}} = (0, 1)$ instead of Q_{stocks}), and explain which token's value vector now receives the most weight.

16.10 Using Section 16.3.2's causal-versus-bidirectional worked example, compute "today"'s own contextual representation (using query $Q_{\text{today}} = (1, 1)$) under a causal mask, noting that a token at the last position of a sequence can attend to every earlier token, and explain why, for the very last token in a sentence, a causal mask produces the same output a fully bidirectional model would.

16.11 Explain, in a short paragraph, why an LLM's hallucination failure mode is harder to catch automatically than a classification model's error rate, referencing Section 16.3.4.

Wulin.Suo@Queensu.ca

16.12 Using Section 16.1.4's method, compute the Gunning Fog index for a passage of your own construction with 30 words, 2 sentences, and 6 complex words, and compare it to both worked paragraphs' scores.

16.13 Compare this chapter's Naive Bayes document classifier (Section 16.2.2) to Chapter 6's original loan-default Naive Bayes discussion: explain, in your own words, why the conditional-independence assumption is more defensible for word-count features in a document than it would be for a firm's financial ratios.

16.14 Using Section 16.3.5's worked example, add a fifth knowledge-base document, Doc5, "Company X's Q3 operating expenses rose 5% to $310 million," with embedding $(0.75, 0.30, 0.00)$, compute its cosine similarity to the same query, and state where it ranks among the five documents.

16.15 Using Section 16.3.6's precision@k example, recompute Precision@4 if Doc5 from the previous exercise (also not relevant to a revenue question) is retrieved fourth, and explain in words why adding a fifth, similarly irrelevant document could never increase precision at any k, only decrease it or leave it unchanged.

16.16 Using Section 16.3.6's faithfulness-score example, an answer embeds at $(0.80, 0.25, 0.00)$. Compute its faithfulness score against Doc1, and state whether it would pass a review threshold of 0.90.

16.17 Write a zero-shot prompt, a two-example few-shot prompt, and a chain-of-thought prompt, following Section 16.3.7's patterns, for the task of classifying whether a sentence from a 10-K risk-factors section describes credit risk or market risk (the task Section 16.2.2 solved with Naive Bayes instead).

16.18 Using Section 16.4.1's worked example, compute x_t for a third diffusion step with $\alpha_t = 0.30$, holding $x_0 = 1.00$ and $\varepsilon = -0.50$ fixed, and confirm that x_t has moved even closer to ε than the two steps already computed.

16.19 Explain, in a short paragraph, why validating a generative AI system requires sampling many outputs rather than checking a single prediction against a labeled test set, referencing Section 16.4.2 and contrasting it with how Chapter 6 evaluates a classification model.

16.20 Pick two of the challenges listed in this chapter and describe a concrete scenario, not already used in the text, in which that challenge could cause an NLP-based financial application to fail despite technically correct code.

16.21 Using the case vignette, explain why AI @ Morgan Stanley Debrief's requirement of client consent before recording is best understood as a governance control (Section 16.4.2) rather than a legal formality, and describe how human review before sending a draft follow-up further limits the hallucination risk of Section 16.3.4.

16.22 Using Section 16.5's Bradley-Terry worked example, recompute the preference probability if the inaccurate summary's reward score r_B rises from 0.8 to 1.7 (holding $r_A = 2.1$ fixed), and explain in words why the preference probability moves toward 0.5 rather than away from it.

16.23 Using Section 16.5's agent example, explain why the agent calling a calculator tool to compute 57.87/3.15 is preferable to the LLM performing that division itself, and describe a different two-step financial task (not P/E) that would similarly benefit from separating retrieval from computation.

16.24 Using Section 16.5's mixture-of-experts example, recompute the active parameter fraction for a layer with 16 experts of 4 billion parameters each, routing each token to its top 4 experts, and compare the resulting total capacity and active-compute cost to the chapter's original 8-expert example.

16.25 Using Section 16.5's quantization worked example, compute the memory footprint of a 175-billion-parameter model at FP16, INT8, and INT4 precision, and state the reduction factor from FP16 to INT4.

16.26 Using Section 16.5's LoRA worked example, recompute the number of trainable parameters and the reduction factor relative to full fine-tuning if the rank r is increased from 8 to 32 (holding $d = 4{,}096$ fixed), and explain in words the trade-off a practitioner faces when choosing a larger rank.

16.27 Using Section 16.5.4's worked example and equations (16.10) and (16.11), (a) compute the task success probability for a fifteen-step task at $p = 0.98$ per step; (b) for $N = 1{,}000$ ten-step tasks at $p = 0.99$, recompute the review load R, the silent failures S, and the precision of the verifier's flags if

Wulin.Suo@Queensu.ca

the verifier is improved to $c = 0.90$ while its false-alarm rate falls to $f = 0.01$; and (c) state which of the two changes, raising c or lowering f, matters more for the review load at this level of p, and why.

16.28 A bank deploys an agent that refreshes know-your-customer files in about twelve steps (retrieve the file, pull registry records, screen each beneficial owner against sanctions lists, compare, draft exceptions). Explain why scoring the agent on its own report that each file was "completed" would overstate its success rate, propose an outcome check on the end state and one trajectory-review rule that would catch a correct file reached by an unacceptable path, and say which tools in the task you would place behind human approval.

16.29 Using Section 16.2.1's event study and Equation (16.4), recompute the day +1 abnormal return, $CAR(-1, +1)$, and its t-statistic if the market had risen 1.5% on day +1 instead of 0.9%, holding every other return and the fitted $\hat{\alpha} = 0.0260\%$, $\hat{\beta} = 1.2949$, and $\hat{\sigma} = 1.4215\%$ fixed. Explain what the result implies about crediting event 5's +2.1% to the call's tone, and how many events with the text's original average CAR of 0.7977% would be needed to reach $t = 2$.

16.30 A private-credit borrower has $300 million of net debt, reported EBITDA of $52 million, add-backs of $9 million (projected synergies) and $4 million (restructuring), and a covenant capping total net leverage at 5.25 times Consolidated EBITDA, with add-backs capped at 20% of EBITDA before add-backs. Using Section 16.3.4's private-credit example, (a) compute leverage on reported EBITDA, with all add-backs, and with the capped add-backs, and state which pass; (b) find the reported EBITDA at which the capped test is breached and the percentage fall from $52 million that this represents; and (c) list the fields an LLM should extract from the credit agreement, with what evidence, before a calculator performs (a) and (b).

16.31 **Challenge (real data).** Download the risk-factors (Item 1A) section of a real company's 10-K from SEC EDGAR's full-text search (free, no login required), and the Loughran-McDonald word list (free from the University of Notre Dame's Software Repository for Accounting and Finance). (a) Apply the tone-scoring method of Section 16.1.3 to the text, reporting the tone score and the positive and negative words matched. (b) Compute the Gunning Fog index (Section 16.1.4) for the same passage, and compare it to both worked-example scores. (c) Real 10-K risk-factor sections are written by legal and investor-relations teams with every incentive to sound cautious whatever the firm's condition; discuss whether a negative tone score on such boilerplate should be read the same way as one on the earnings-call language of this chapter's worked examples.

16.10 Further Reading

- Loughran and McDonald (2011), "When Is a Liability Not a Liability? Textual Analysis, Dictionaries, and 10-Ks." The original paper introducing the finance-specific sentiment dictionary used in Section 16.1.3.
- Hansen and McMahon (2016), "Shocking Language: Understanding the Macroeconomic Effects of Central Bank Communication." A research-grade version of Section 16.1.3's central-bank worked example, using text methods to separate what the Federal Reserve says from what markets expected it to say.
- Jurafsky and Martin (n.d.), *Speech and Language Processing*. A comprehensive, freely available reference covering bag-of-words, TF-IDF, and embedding methods (Sections 16.1.2 and 16.2.3) in far greater depth.
- Vaswani et al. (2017), "Attention Is All You Need." The original transformer paper introducing the self-attention mechanism worked through by hand in Section 16.3.1.
- Mikolov et al. (2013), *Efficient Estimation of Word Representations in Vector Space*. The original Word2Vec paper underlying the embedding geometry illustrated in Section 16.2.3.
- Li (2010), "The Information Content of Forward-Looking Statements in Corporate Filings—A Naïve Bayesian Machine Learning Approach." An application of the Naive Bayes text classification approach used in Section 16.2.2 to real corporate filings.

Wulin.Suo@Queensu.ca

- Lewis et al. (2020), "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." The original paper introducing the retrieval-augmented generation approach worked through in Section 16.3.5.
- Wei et al. (2022), "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." The original paper on the chain-of-thought prompting strategy discussed in Section 16.3.7.
- Goodfellow et al. (2020), "Generative Adversarial Networks." The original paper introducing the generator-discriminator framework discussed in Section 16.4.1.
- Ho et al. (2020), "Denoising Diffusion Probabilistic Models." The paper underlying the forward and reverse diffusion process worked through by hand in Section 16.4.1.
- Board of Governors of the Federal Reserve System and Office of the Comptroller of the Currency (2011), *Supervisory Guidance on Model Risk Management*. The foundational U.S. model risk management guidance, superseded in April 2026 by SR 26-2 (Federal Reserve, OCC, and FDIC 2026), which keeps its pillars but leaves the generative AI governance questions raised in Section 16.4.2 outside its scope.
- Morgan Stanley (n.d.), *"Key Milestone in Innovation Journey with OpenAI" and "Launch of AI @ Morgan Stanley Debrief"*. The primary source for the retrieval-augmented generation and meeting-summarization deployment described in this chapter's case vignette.
- Ouyang et al. (2022), "Training Language Models to Follow Instructions with Human Feedback." The paper introducing the RLHF pipeline and reward modeling worked through by hand in Section 16.5.
- Yao et al. (2023), "ReAct: Synergizing Reasoning and Acting in Language Models." A foundational paper on the agent tool-use loop illustrated in Section 16.5's worked example.
- Yao et al. (2024), *τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains*. The agent benchmark discussed in Section 16.5.4, which scores an agent on the end state of each task and introduces the pass^k measure of reliability across repeated attempts.
- Shazeer et al. (2017), "Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer." The original paper introducing the sparse routing architecture discussed in Section 16.5.
- Wu et al. (2023), *BloombergGPT: A Large Language Model for Finance*. The paper describing the domain-adapted financial language model discussed in Section 16.5.
- Islam et al. (2023), *FinanceBench: A New Benchmark for Financial Question Answering*. The benchmark of filing-based financial questions discussed in Section 16.3.6, and a practical template for building an institution's own evaluation set.
- SEC (1998), *A Plain English Handbook*. The SEC's own guidance underlying the readability and disclosure-analysis discussion in Section 16.1.4.

Wulin.Suo@Queensu.ca